Faculty of Engineering & Technology

Electrical & Computer Engineering Department

REAL-TIME SYSTEM AND INTERFACING TECHNIQUES LABORATORY – ENCS5140

Report1

Experiment 1&2&3: Introduction to Arduino, UART in Arduino and I2C in Arduino

---

Prepared by:

Ahmed Zubaidia 1200105

Partners:

Jana Herzallah 1201139

Lana Badwan 1200071

Instructor: Dr. Hanna Bullata

Teaching Assistant: Eng. Rania Shahwan

Section: 5

Date: 30/10/2024

Abstract

This report aims to summarize the key objectives of the first three experiments; starting with the first one it aimed to highlight the concept of light detection and responsive control through hardware and software interrupts which will be implemented to react instantaneously to particular events. Moving to the next experiments, the main objectives were to understand how the Arduino can send and receive data to / from multiple devices such as the computer itself or another Arduino through the concept of the UART physical circuit or the I2C communication protocol. In addition to sending and receiving from components connected to the Arduino such as LDR, potentiometer, push button, temperature sensor and displaying some texts on the LCDs. With the use of Arduino IDE, breadboards, Arduinos, serial monitor and the mentioned components, the sending and receiving processes were successfully implemented and understood as will be shown in the next sections.

Contents

List of Figures

List of Tables

Theory

Experiment 1

What is Arduino?

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs like light on a sensor, a finger on a button and convert it into an output. We can tell our board what to do by sending a set of instructions to the microcontroller on the board. We use the Arduino programming language (based on Wiring), and the Arduino Software (IDE), based on Processing.

Over the years Arduino has become the brain of many projects, from everyday objects to complex scientific instruments. Around this open-source platform, a global community of creatives - students, artists, programmers, and professionals - have come together and they have contributed an amazing amount of easily accessible knowledge that may be useful for beginners and experts as well. [1]

Arduino Uno

The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller (MCU) and developed by Arduino.cc and initially released in 2010. The microcontroller board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable. It can be powered by a USB cable or a barrel connector that accepts voltages between 7 and 20 volts, such as a rectangular 9-volt battery. It has the same microcontroller as the Arduino Nano board, and the same headers as the Leonardo board. The hardware reference design is distributed under a Creative Commons Attribution Share-Alike 2.5 license and is available on the Arduino website. Layout and production files for some versions of the hardware are also available.[2]

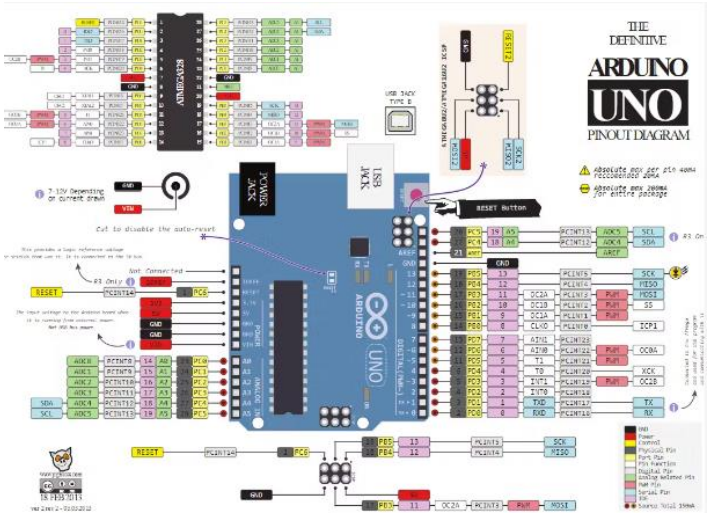

Figure 1:Arduino Uno board [3]

Why should we get an Arduino?

Arduino is used to build an independent device that can connect and interact with other hardware and software devices. It's also lets us to control things Such as reducing motor pressure or lowering the curtains based on the amount of light in the room, a light sensor can detect and send information to the Arduino. It can also read an information from several sources like a keyboard and a webpages and then turn that into actions like switching on a light or showing what you write on a screen.

With Arduino it is possible to automate anything to make autonomous agents to control any devices or anything else you can think of to control light and devices. We can go for an Arduino-based solution, especially in developments of devices connected to the Internet.

Arduino is a technology that has a fast-learning curve with basic knowledge of programming and electronics, which allows developing projects in the field of Smart Cities, the Internet of Things, wearable devices, health, leisure, education, robotics, etc… [4]

Photocells (Photoresistors, LDR (light dependent resistor)

A photocell is also known as an electron tube, photoelectric cell, electric eye, and phototube. This is an

electronic instrument that is mainly react to light it can generate and control electric current levels. It is also called a resistor basically because its resistance value depends on the amount of light that falling on it.

This device functions on the principle of semiconductor photoconductivity which means that when light particles (photons) hit the semiconductor it allows electron movement and thus reducing the resistance level. [5]

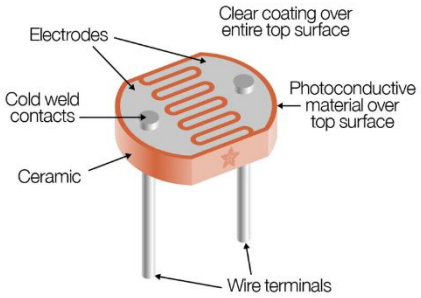

Figure 2:Photocell terminals [6]

How to measure light using a photocell?

A photocell's resistance changes as the face is exposed to more light. When its dark, the sensor looks like a large resistor up to 10MΩ, as the light level increases, the resistance goes down. This graph indicates approximately the resistance of the sensor at different light levels. [7]

The easiest way to measure a resistive sensor is to connect one end to Power and the other to a pull-down resistor to ground. The connection point between the fixed pulldown resistor and the variable resistor of the photocell is linked to an analog input on a microcontroller, like an Arduino (as shown).

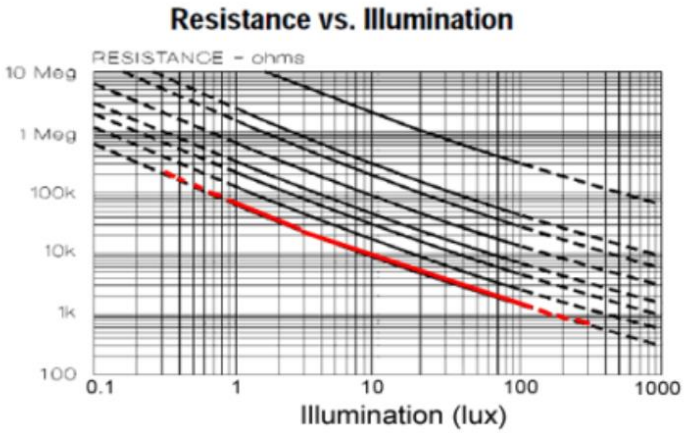

Figure 3:Photocell resistance vs illumination [7]

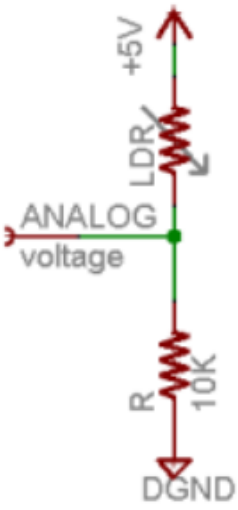

Figure 4:Figure 3:Photocell resistance connection [7]

$$Vo = Vcc\ (\frac{R}{R + Photocell})$$

The equation shows that the voltage is proportional to the inverse of the photocell resistance.

Arduino Interrupts

An interrupt in relation to microcontrollers is a mechanism that temporarily suspends the main program, passing control to an interim code.

In terms of Arduino this means that a response code associated with a specific software situation or hardware change is executed while the standard code described inside the programming loop (i.e. loop() function) is suspended. The interrupt service routine (ISR) is a well-organized section of code that defines this response .[8]

interrupts are classified into two categories.

1. **Hardware interrupts:** generated by the microcontroller's input/output pins. An external hardware/component triggers a change in voltage signal for Arduino's input/output pin.

In Arduino, there are two types of hardware interrupts: external and pin change interrupts.

2. **Software interrupts**: generated by user-defined program instructions. They're always associated with the microcontroller's built-in peripherals and communication port. These interrupts are not triggered by an external hardware component but by changes to the built-in peripherals or software configuration. [8]

Arduino Timer Interrupts

Timer interrupts in Arduino pause the sequential execution of a program loop() function for a predefined number of seconds (timed intervals) to execute a different set of commands. After the set commands are executed, the program resumes again from the same position. The Arduino comes with three timers known as Timer0 (8-bit timer), Timer1 (16-bit timer), and Timer2 (8-bit timer). They act as a clock and are used to keep track of time based events. These timers will be programmed using registers.[9]

How do Timer Interrupts work?

Hence Timer1 is a 16-bit timer and Timer2 is an 8-bit timer, Timer1 can count from 0-65537 while Timer2 counts from 0 to 255. Based on the chosen timer mode, the timer will increment its value until it hits its maximum limit

and then reset to 0. This generates a triangle-shaped waveform that the timer follows. Consequently, an interrupt will trigger as the value ascends from 0 to 65537 in the case of Timer1, before returning to 0 and starting the cycle again.[9]

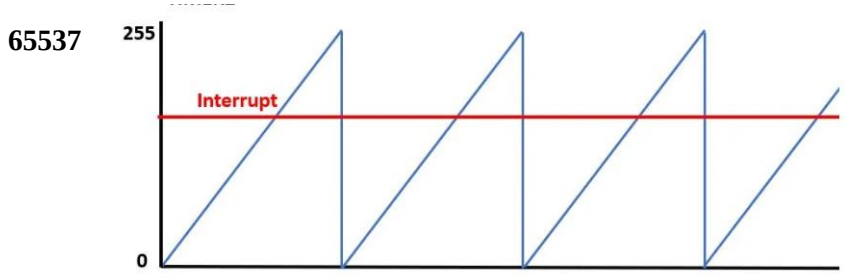

Figure 5:Timer1 [9]

Each of the three timers are able to generate different interrupts.

Timer1 is capable of generating interruptions for Compare Match, Overflow, and Input Capture. An interrupt is activated in the case of a compare match when the timer's current value in the register matches the specified compare value. For instance, if we set the compare value to 50, an interrupt will be triggered each time Timer1 reaches that value. The next kind of interrupt is the overflow, which occurs when the timer reaches its maximum limit. Lastly, there is the input capture interrupt, which saves the timer value into a separate register whenever an external trigger takes place.[9]

Timer Speed

One important aspect of the timer clock is its speed at which it increments the counter. It is directly related to the prescaler we will set for our timer. The prescaler can take values from 1, 8, 64, 256 or 1024 which is then used to divide the maximum Arduino clock speed i.e 16MHz to find the maximum timer speed.

$Timer\ speed\ (Hz) =$ Arduino clock speed (16MHz) / prescaler .**[9]**

For prescaler with value =1

the timer speed calculated as shown below:

Timer speed (Hz) = Arduino clock speed (16MHz) / 1

= 16MHz

This means that timer one takes 1/16,000,000 seconds, or 0.0625 microseconds, for the register to increase by one. This indicates that 0.0625 micro * 65535 = 0.004096 seconds are needed for the timer to reach its maximum value.

Registers for Timer1

Key registers you'll use to interact with Timer1 include:

- TCCRx: This is the Timer/Counter Control Register.
- TCNTx: It holds the current timer value.
- OCRx: The Output Compare Register.
- ICRx: Input Capture Register.
- TIMSKx: This is the Timer/Counter Interrupt Mask Register, which lets you enable or disable timer interrupts.
- TIFRx: The Timer/Counter Interrupt Flag Register.

As we already know Timer1 is able to generate Output Compare Match, Overflow and Input Capture interrupts. For Output Compare Match enablement, OCIE1B and OCIE1A bits are used. Likewise, for overflow interrupt enablement, TOIE1 bit is used. Similarly, we use ICIE1 bit for input capture interrupt.

Timer1 consists of two major registers TCCR1A and TCCR1B which control the timers where TCCR1A is responsible for PWM and TCCR1B is used to set the prescalar value. We will set all the bits in the TCCR1A register to 0 as we will not be using it in this experiment.

Table 1:TCCR1A Timer1 Control Register A[9]

Bit	7	6	5	4	3	2	1	0
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

However, for TCCR1B, the first three bits are used to set the prescalar value. These are known as

CS10, CS11 and CS12 bits.

Table 2:TCCR1B Timer1 Control Register B[9]

Bit	7	6	5	4	3	2	1	0
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10
Read/Write	W	W	R	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

To set the operating mode, refer the table below:

Table 3:Timer one modes

WGM13	WGM12	WGM11	WGM10	Mode	TOP
0	0	0	0	Normal	0xFFFF
0	1	0	0	CTC	OCR1A

Normal Mode:

All WGM bits are set to 0 (`WGM13 = 0, WGM12 = 0, WGM11 = 0, WGM10 = 0`).

In this mode, the timer simply counts up from 0 to the maximum value (0xFFFF for a 16-bit timer, which is 65535 in decimal) and then overflows, resetting back to 0.

CTC Mode (Clear Timer on Compare Match):

WGM bits are configured as `WGM13 = 0, WGM12 = 1, WGM11 = 0, WGM10 = 0`.

In CTC mode, the timer counts up to a specific value set in the **Output Compare Register (OCR1A)**, then resets to 0.

This mode is commonly used to generate precise timing intervals or frequencies, as the timer will reset at the compare match rather than counting all the way to 0xFFFF.

Experiment 2

UART (Universal Asynchronous Receiver/Transmitter)

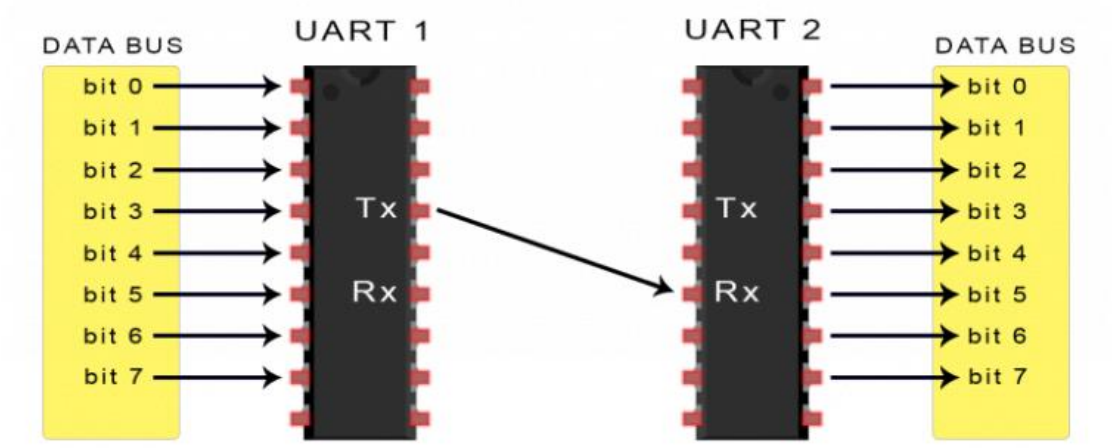

Figure 6: example of UART TX and Rx pins

UART, refers to Universal Asynchronous Receiver/Transmitter, which is a hardware framework or design in (IC) integrated circuit and hardware for serial data communication. UART is not like other protocols like I2c a communication protocol, than it makes the transmission and reception of the data easier between the two UARTs directly. The main functionality that UART do is to convert the parallel data from a controlling device for example CPU, to a serial from transmission, and the same for receiving, UART converts incoming serial data back to parallel form for the receiving device. In UART we just need two wires to communicate TX (Transmit) and RX(Receive)[11]

In UART communication, the data transmitted asynchronously, which mean that there is on clock singles for synchronize data between the devices, but we achieve it by using start and stop bits. UART add these bits in the transmitting to mark or know the beginning and the end of each packet, so the receiving UART can identify start and stop reading. Also the transmission relies of a predetermined baud rate, to maintain timing between the both UARTs. [11]

A UART data packet consists of four main components: the start bit, data frame , parity bit , and stop bits. To initiate data transfer, the transmitting UART sends a start bit by pulling the transmission line from high to low voltage for one clock cycle, which tell the receiving UART to start reading at the set baud rate, and the data frame contains the actual data, usually between 5 to 9 bits length, depending on parity bit if it used or not. The parity bit
provides a way to verify data integrity. By sum the odd and even bits in the data frame, the receive can detect and know if there are any errors due to noise, mismatched baud rates, or interference. Finally, stop bits signal the end of transmission comes by returning the voltage to high, which is mean the end of the data packet.[11]

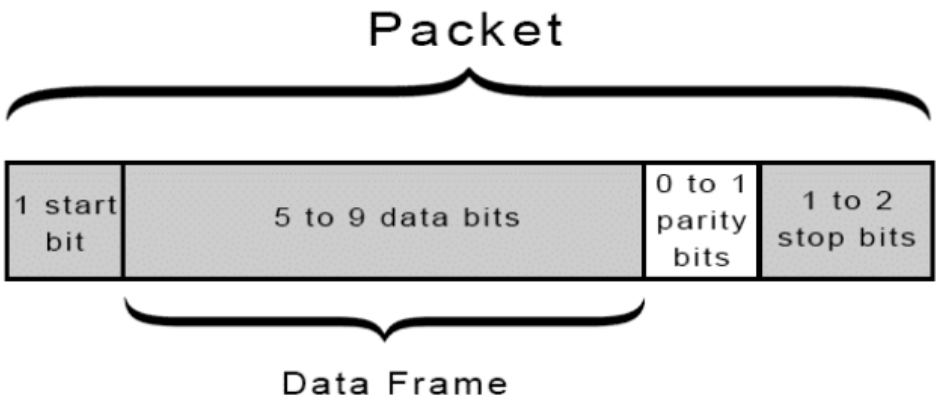

Figure 7: Packet diagram [1].

Serial Communication with Arduino (Tx, Rx)

In Arduino, serial communication with external devices or computers via TX (Transmit) and RX (Receive) pins allows debugging and controlling peripherals. These pins operate at TTL logic level 5V or 3.3V depending on the Arduino model. Avoid connecting these pins directly to RS232 serial ports that operate at higher (+/-12V) voltages as this will damage the Arduino board. Many Arduino boards include at least 1 serial port (digital pins 0 (RX) and 1 (TX), at times called UART (Universal Asynchronous Receiver / Transmitter) or USART (Universal Synchronous / Asynchronous Receiver / Transmitter). These pins are engaged when communicating with a computer via USB, and thus are not available for general digital input or output during active serial communication.[12]

Serial communication is simplified with Arduino's built-in functions. The Serial.begin () function initializes serial communication at a given baud rate and may optionally set data bits, parity, and stop bits. This initialization is usually passed in to setup (). Configurations include SERIAL _ 5N1 (five data bits, without parity, one stop bit) along with SERIAL _ 8N1 (8 data bits, no parity, one stop bit), etc.[12]

For data reception Arduino provides several methods. The Serial.available () function returns the number of bytes available in the receive buffer for incoming data. Serial.read () reads the 1st byte of new serial data, and Serial.readBytes (buffer, length) reads several bytes into a buffer until the desired length is reached or a timeout happens. For reading strings, Serial.readString () takes characters from the serial buffer until a timeout. Also, Serial.parseInt (Serial.parseFloat and) () return integer and floating-point numbers from serial data, respectively.[12]

Also Serial.print () sends data in human-readable ASCII text (for numbers and text). On the other hand, Serial.write () sends raw binary data as a byte or sequence of bytes, therefore it's ideal for delivering binary commands. Together these functions provide robust and flexible serial communication options for Arduino supporting various data transmission and reception requirements.[12]

Experiment 3

Part1: Overview of I2C Protocol

I2C communication scheme transfers the data bit by bit along the bus and its considered synchronous since the output is synchronized to the sampling of bits by a clock signal shared between the master and the slave.

The I2C communication protocol combines the best features of other serial communication protocols such as SPI and UART. It’s mostly invented to solve the problem of the systems that consists of multiple devices which each of them can be executing in different pace. So we need to make the faster ones having more throughput than the slower ones with the idea of having the minimum cost

possible.

As shown in figure [6], there are two wires in the system that communicates through I2C which are SDA which will carry the data to be transmitted and to be received and SCL that will represent the clock signal.

These lines have the ability to carry the signals in both directions and since its connected to the VCC its high when the lines are not being used up.

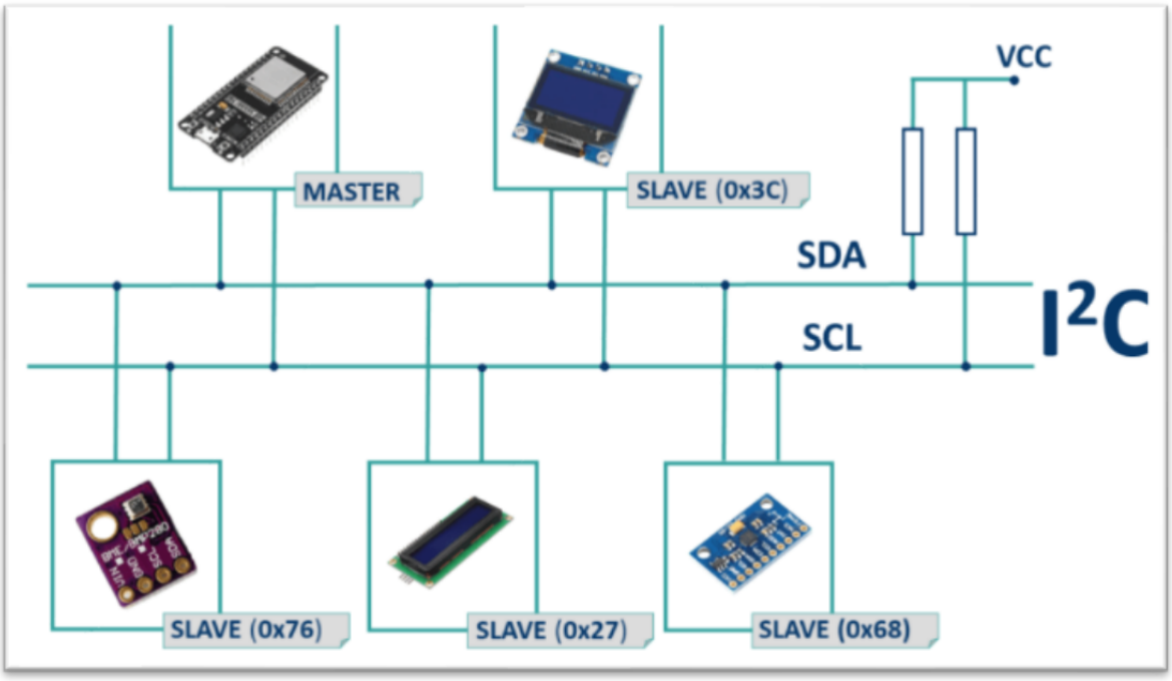

Figure 8: I2C Main Wires used to communicate devices

Part2: I2C data transfer and timing

1. **Bit transfer**

   SDA signal can be changed when the SCL signal is low, and only one bit can be transferred during the clock pulse (when the clock goes high).[15]

2. **Start and stop conditions**

   The I2C communication should start with a start condition and gets ended by a stop condition. The start sign is usually considered as the transition of SDA from high to low while the transition from low to high status is considered as sign of stop condition. Whenever the communication is started after a start condition the bus is considered as a busy line and it

can be used only after the stop condition is found and the communication is ended. The stop conditions and start conditions can be noticed according to the next figure:

-

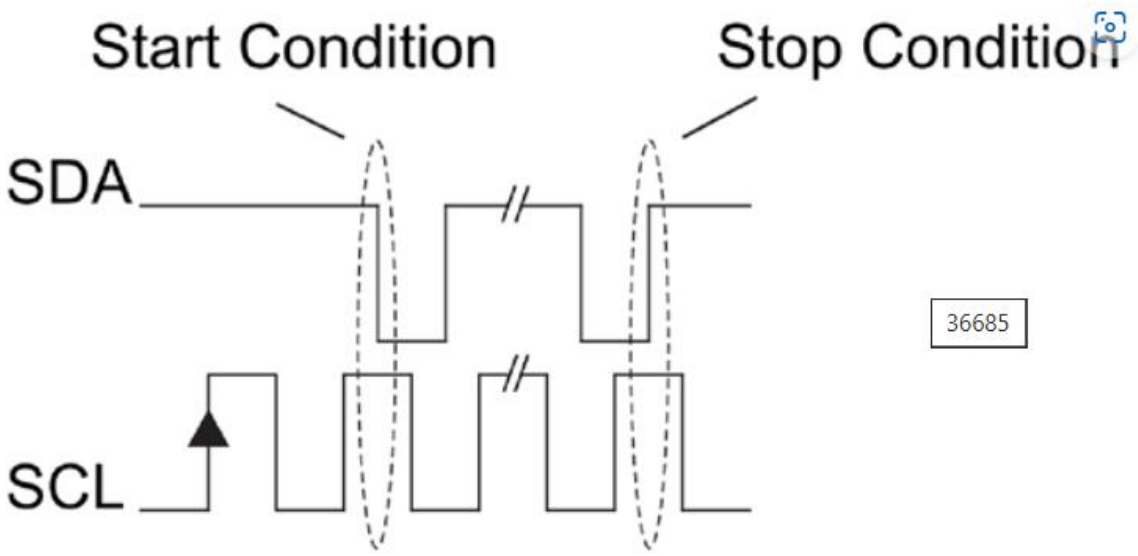

Figure 9 : start and stop conditions

3. I2C data transfer

The data carried via I2C bus length is 8 bits or byte. An acknowledgment bit should follow each byte. It's used to check if the device is ready to continue communicating or not. While the data is being transferred over the serial data line (SDA), the serial clock line (SCL) should generate pulses for the clock to make the process synchronized.

Whenever a no acknowledgment signal is received the master either generates a stopping condition to end the communication or generating a repeated start condition which is a unique case that takes place when the start condition is transmitted without detecting a stop condition for the first start condition, it can be beneficial for when changing the direction of data transfer, retrying transferring data. [13]

4. I2C SYNCHRONIZATION

When a master makes the SCL low, it should stay in the low state till the masters put it off or release it and then they all return it to high. That's crucial because the data is only changed whenever the clock is low so we only want one master to change data at the same time to hold the synchronization.

5. ARBITRATION

If there is more than one master trying to initiate a connection with the peripheral through the I2C bus, there is an arbitration procedure that should be done to determine which master will win. This procedure should be done while the SCL is high. And the masters can know who won the arbitration by checking the generated SDA with the one in the bus. When the data bit should be one and it's found to be zero that means that another master has won

6. COMMUNICATION WITH 7-BIT I2C ADDRESSES

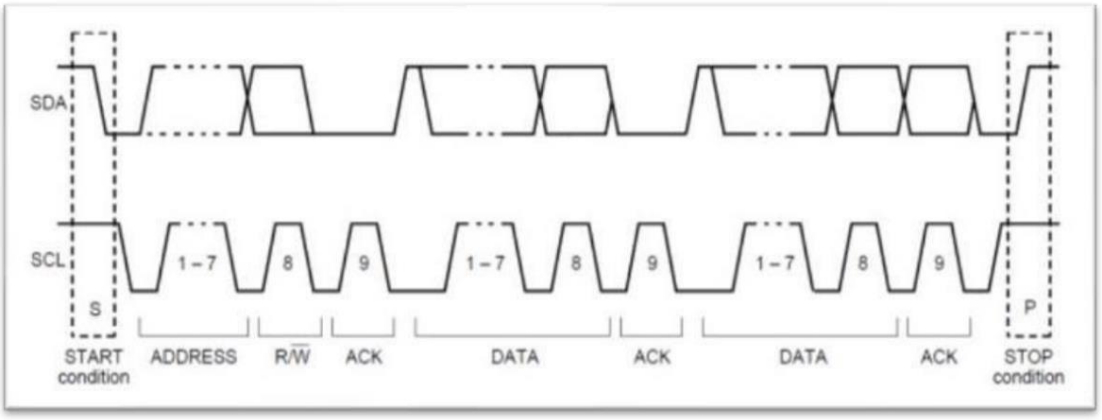

Figure 10: communication of I2C devices signals

According to the above figure, the I2C communication starts with a start condition which is a change in SDA when SCL is high, then its followed by 7 bits that represent the address of the I2C , then its followed by a zero if the process is writing and 1 if the process is reading. Each byte is followed by Acknowledgment bit. Then the master can place a stop bit if the I2C communication is done. While it also can generate a repeated start bit to change the address of the I2C if it wants to communicate another peripheral or if it wants to change the reading to writing or the opposite

Part3: LM75B temperature sensor

The temperature sensor measures temperatures from -50 degrees to +125 degrees [15]. The key principle of the sensor reading is that the Vbe which is the forward voltage in the silicon diode is related to the temperature surrounding it [7]. The relationship between the Vbe and temperature is shown according to this equation:

$$\Delta VBE = KT \cdot \ln(IC2IC1)/q$$

Where K is the Boltzmann constant, T is the absolute temperature in Kelvin and q is the charge of an electron.

When IC2 and IC1 which are the collector currents in the diodes on the transistor board are not equal, that will give a value to the change in VBE, otherwise no change will occur in it.

The LM75B configuration is explained in the figure below:

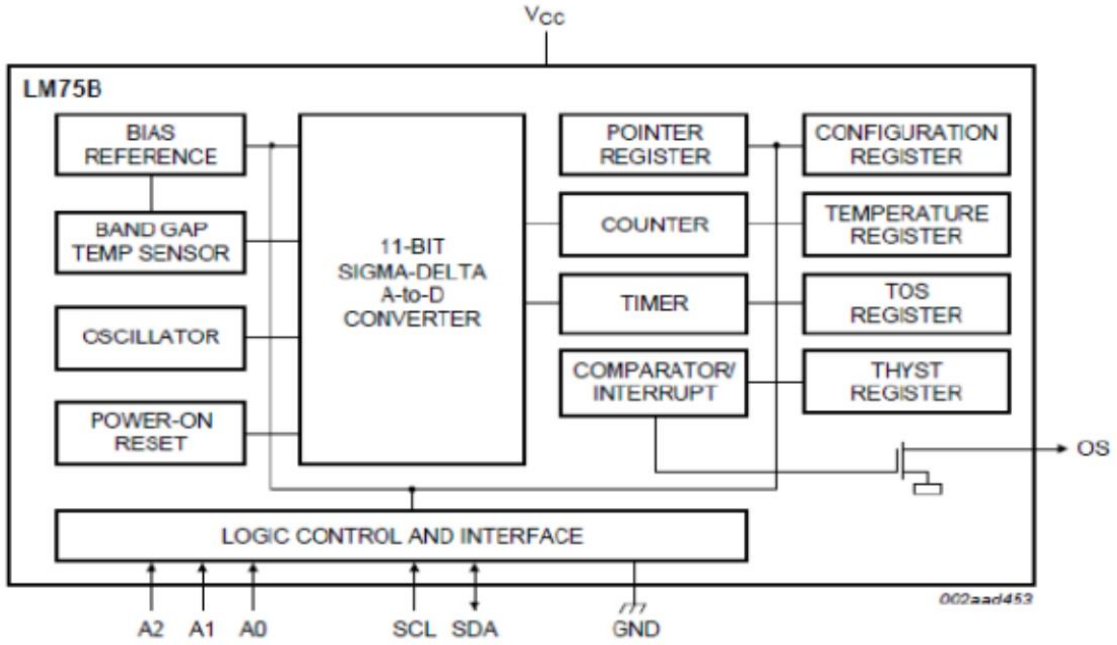

Figure 11: Configuration of temperature sensor

As shown in figure [9], there is an 11-bit Sigma-delta A-to-D convertor which writes the result of the conversion to the I2C communication.

The above configuration is usually abstracted into an integrated circuit as shown in the figure below which has main peripherals summarized in this table:

Pin No.	Symbol	Description
1	SDA	Bidirectional Serial Data
2	SCL	Serial Data Clock Input
3	INT/CMPRT	Interrupt or Comparator Output
4	GND	System Ground
5	A_2	Address Select Pin (MSB)
6	A_1	Address Select Pin
7	A_0	Address Select Pin (LSB)
8	VDD	Power Supply Input

Table 4 : peripherals of LM75B

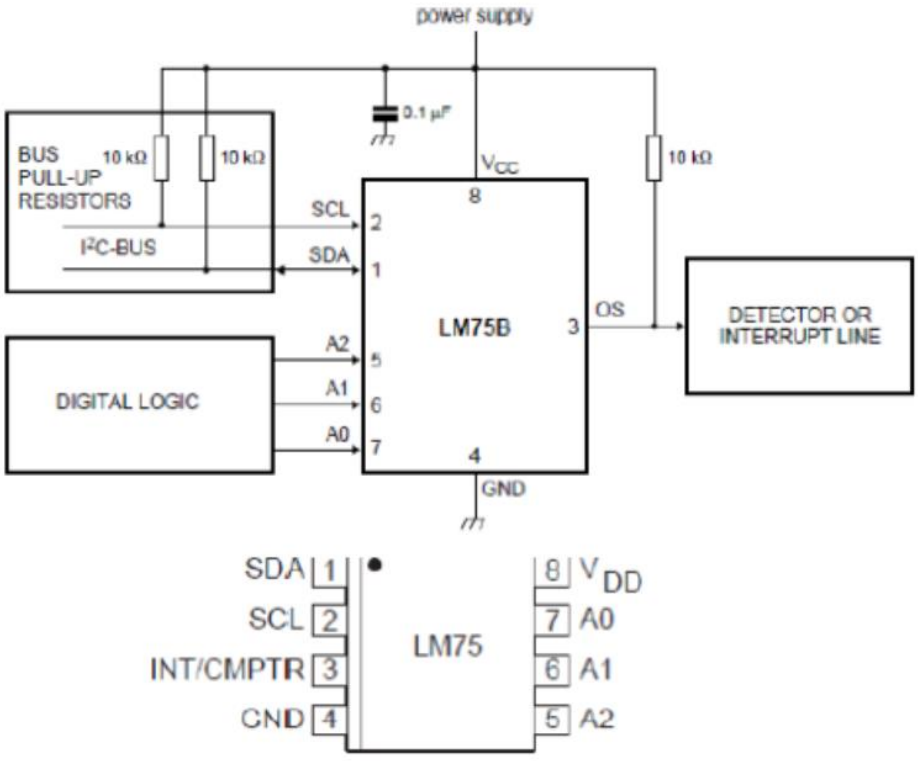

Figure 12: Temperature sensor IC

The address of I2C communication the LM75B is 7 bits long where the most significant bits are set to 1001 and the least significant bits are A2 A1 A0. that allows for multiple configurations of multiple sensors to be connected to an I2C bus. Changing the values of the least significant bits will generate a unique address of each sensor to be contacted by the master at. These addresses can go from 0x48 till 0x4F[**AI**].

Procedure, Results Discussion and TODOs

Experiment 1

Photocells

In this part, the LDR was connected to the Arduino analog input pin A0 through a pull down **resistor**. This resistor forms a voltage divider with the LDR this producing a variable voltage at the analog input depending on the light level. The second terminal was connected to 5v on Arduino board to provide power.

The Led was connected through current limiting resistor to digital pin 11 which is a pulse width modulation (PWM) pin on the Arduino that allows the Arduino to vary the led brightness depends on the reading value from the photocell. The kathode terminal was connected to the GND as in the below figure.

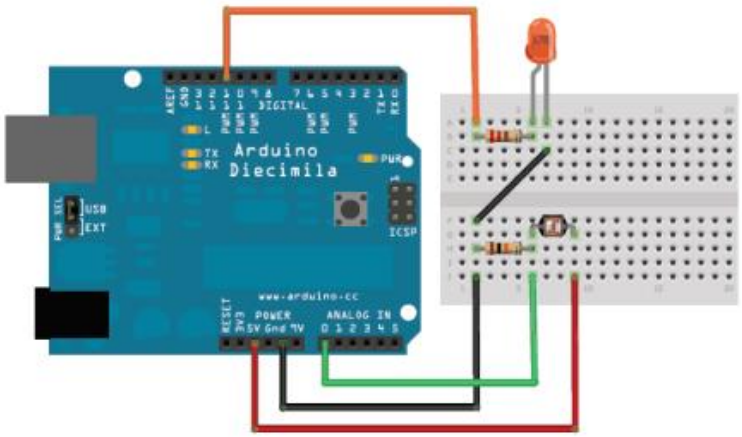

Figure 13:Table 4:Circuit diagram for Photocell connection with Arduino board[7]

The software procedure was started by writing a code that gradually increases the LEDs brightness as the surrounding light level decreases and gradually decreases the LEDs brightness as the surrounding light level increases.

As the surrounding light gets darker the **photocellReading** become lower while the photocell's resistance increases. We obtained the photocell reading by reading from analog pin A0:

photocellReading = analogRead(photocellPin);

To invert this reading, we converted the low photocell reading to a higher value by subtracting it from 1023:

photocellReading = 1023 - photocellReading;

This provided a new higher photocell value. We then mapped this higher analog value to a corresponding high digital value, which increased the LED's brightness

LEDbrightness = map(photocellReading, 0, 1023, 0, 255);
analogWrite(LEDpin, LEDbrightness);

As the surrounding light gets brighter, the **photocellReading** increases, while the photocell's resistance decreases.

When applying the previous formula :

photocellReading = 1023 - photocellReading;

Then we inverted the high **photocellReading** to a lower value which reduced the LED brightness.

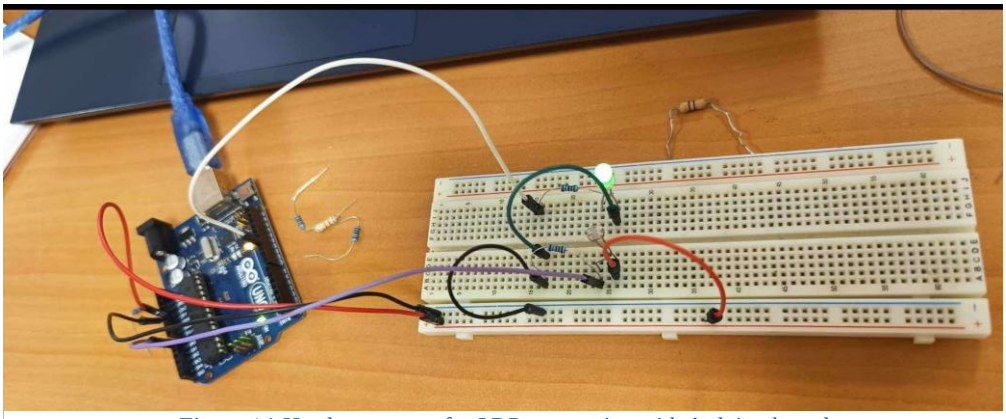

Figure 14:Hardware setup for LDR connection with Arduino board

Finally, the output matched the expected result. The LED becomes brighter as the sensor detects darker surroundings and it becomes darker as the sensor detects brighter surroundings and that was done as in the video in link.

photocell example_1.mp4

Hardware Interrupt

In thus part, we were connected a push button to digital pin 2 on Arduiono as a hardware interrupt; This is done to control the LED which was connected to digital pin 13. The connection of this part shown in the image below.

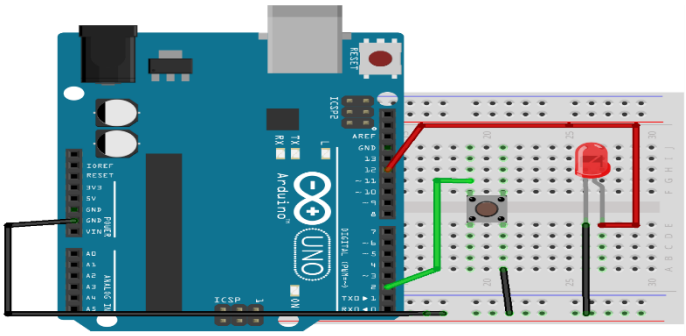

Figure 15:connection for hardware interrupt part.[10]

The software procedure was started by writing a code that toggles the LED state through the push button (Hardware intterupt), The **attachInterrupt()** function was used to attach an interrupt to digital pin 2 which corresponds to interrupt 0 on the Arduino Uno. We were put the mode of interrupt to be a **CHANGE** mode,, this specifies that the interrupt should trigger whenever the button state changes. Whether it is pressed or released.

We were call **blink()** function when interrupt occurred to toggle the Led state.

The code contains a loop function which is write the Led state Pin 13 digital Write(pin, state) after a change applied on a button whether it is falling or rising change.

In this part the output at first did not match the expected outcome! When we pressed the button we faced a problem called the "bouncing" effect. Due to tiny vibrations mechanical buttons can trigger

several fast transitions leading to multiple interruptions that are not intended. With each button press this "bouncing" action made the LED flicker rather than toggling smoothly. The bouncing effect was displayed with the video in the link below.

Hardware Interrupt with bouncing effect.mp4

To solve this issue we were implemented a debounce mechanism. we used the `millis()` function to track the time between interrupts. By checking if a certain amount of time had passed since the last interrupt we could ignore quick repeated triggers and ensure only one interrupt is registered with each button change. After this done then the actual output be similar as expected one. The result shown in the video attached below.

debounce effect.mp4

software Interrupt

In this part, a timer overflow interrupt on Timer1 is used to toggle a LED on pin 13. When the timer reaches its maximum value and then overflows back to zero, an overflow interrupt is triggered. with set up an interrupt we can make the LED toggle at a consistent interval without using **delay()** function allowing other code in the loop() function to run without waiting till the end of ISR. The circuit connection of this part shown in the image below.

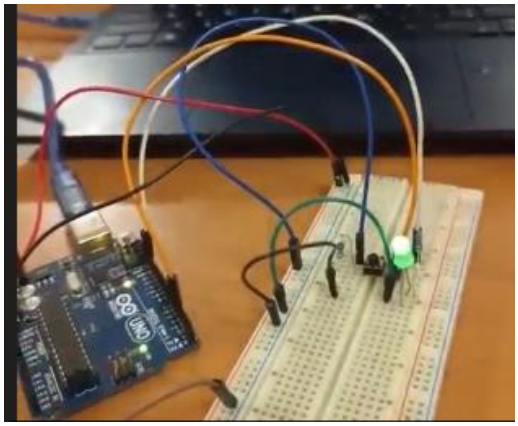

Figure 16:Timer1 circuit connection

The software level was started by writing a code which is toggling a led every 0.5 second by using timer one, which is a software interrupt that solve the problem of using delay function while it let the while loop to complete running and did not getting stuck on the ISR function.

At first, we disabled all interrupts by calling this function **noInterrupts(); ,**then we initialize TCCR1A and TCCR1B register with 0 value, after that the prescaler was determined with 256 value and this is done by setting the first three bits of TCCR1B like this CS00 = 0, CS01 =0 , CS02 =1 and this is the line in the code that was used to set the prescaler to be equal 256, TCCR1B |= (1 << CS12).

After the overflow mode was enabled by editing **TIMSK1** register by writing this line **TIMSK1 |= (1 << TOIE1);**. Then we were defined the interval that was needed for timer one to toggling the led each 0.5 second. And this preload value was calculated as shown below.

At first we calculated the time taken by the timer to be incremented by one.

Timer speed = 16MHz/prescaler.

= 16MHz/256.

=62500Hz.

Time needed to increment Timer one register by one= 1/625500

=0.000016 second.

Now to calculate the number of increments needed to make an interrupt with timer one each 0.5 second we use this formula.

1 increment -------→ 0.000016 sec

X increments -----→ 0.5 sec

0.000016 X = 0.5

X = 0.5/0.000016

= 31250.

This means we need to increment Timer1 by 31250 to make an interrupt each 0.5 second.

Then the preload value was calculates as shown below,

Preload = 65536 -31250

= 34286

Then we set TCNT1 to be equal the preload value to make 31250 increments which are from TCNT1 value to the maximum value for timer one (65536).

We were enabled interrupts by using this function **interrupts();** Then the ISR function was called each 0.5 second to toggle the Led by using an xor logic.

digitalWrite(ledPin, digitalRead(ledPin) ^ 1);

Finally, The output for this part was matched the expected one. The result shown in the video in the link below and the led was toggled every 0.5 second by using Timer1 interrupt.

Timer1example3.mp4

Experiment 2

Part1: Basic communication between Arduino & PC

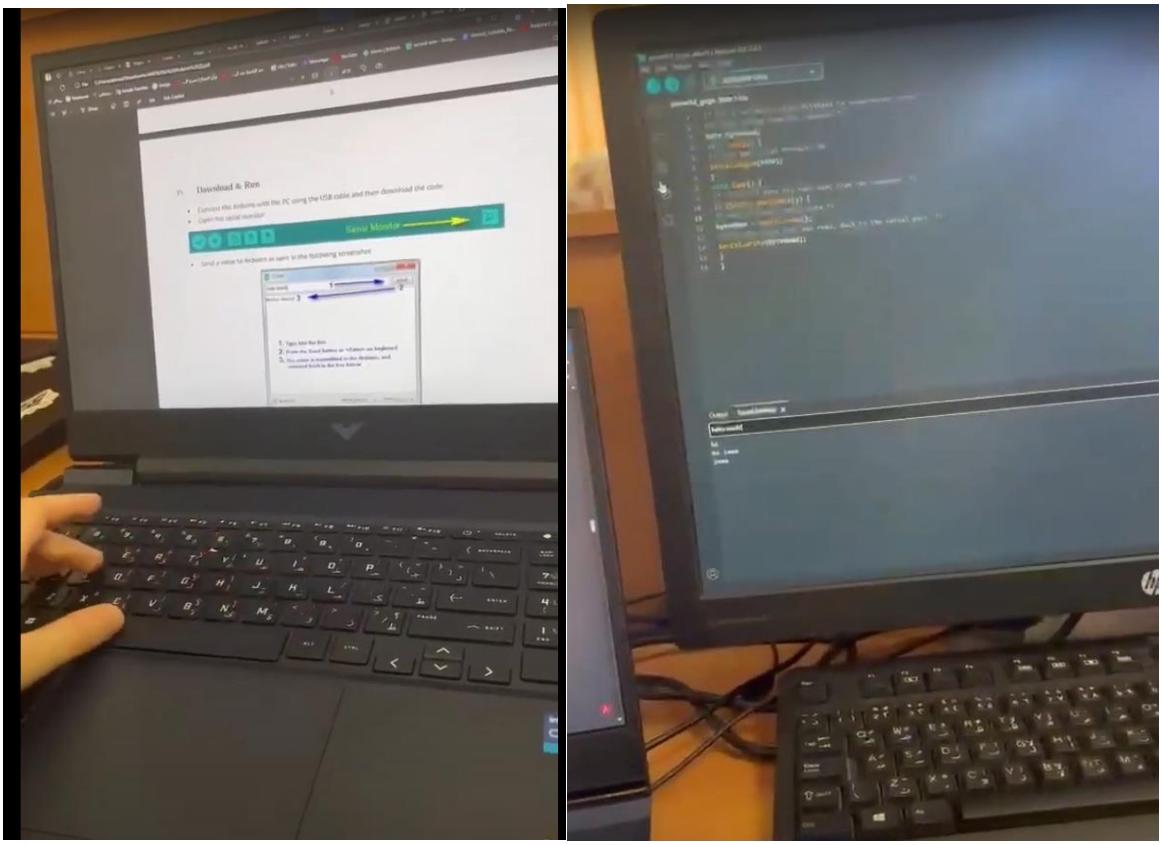

Figure 17: Circuit basic communication between Arduino & PC

The goal of this part is to start a simple serial communication connection between an Arduino and a pc. We will use this setup to send and receive a simple message, "Hello World!", In order to be familiar with the setup required for serial communication.

Required Components:

- PC: For running the Arduino IDE and monitoring serial communication.
- Arduino UNO Board: To send and receive serial data.
- USB Cable: To connect the Arduino to the PC, allowing serial communication via the

Circuit Setup:

First, we will connect the Arduino UNO board to the pc using the USB cable. It allows and enable serial communication.

Results and Discussion Experiment 1

After the code was uploaded to the Arduino and the Serial Monitor was opened in the Arduino IDE, information like "Hello World!" were typed into the Serial Monitor and then sent by using Serial.write() to the Arduino. The data was received successfully by the Arduino with Serial.read() and sent to the Serial Monitor, showing that communication was going as expected and worked.

This experiment demonstrates that basic serial communication between a PC and an Arduino it can be easy to create, also it was observed that both the Arduino and PC must use the same baud rate, otherwise, data may not appear correctly.

Part2: Basic communication between 2 Arduinos

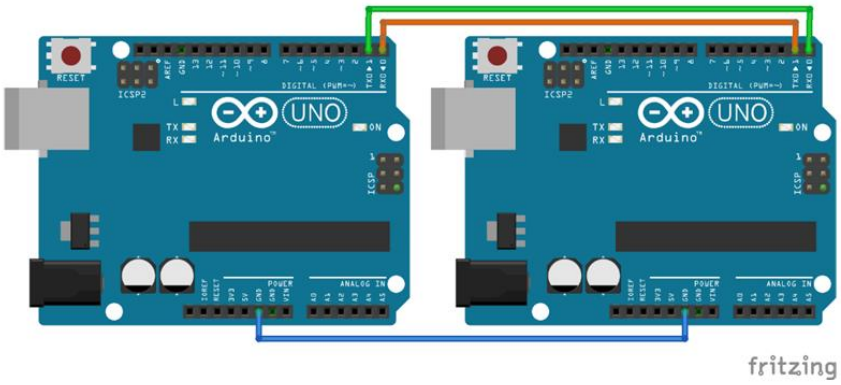

Figure 18: Connecting 2 Arduinos using TX & RX

In this part of the experiment, it involves connecting two Arduino boards in serial communication. This setup will let one of the Arduinos functions as the sender and the other the receiver.

Circuit Setup:

- Connect the TX (Transmit) pin of 1st Arduino with the RX (Receive) pin of 2nd Arduino.
- In the same manner, connect the RX pin of the first Arduino to the TX pin of the 2nd Arduino (TX1 RX2 and RX1 TX2).
- Make sure that there is a common ground shared between the two Arduinos to finish the circuit.

Results and Discussion for experiment 2:

Observed Result:

The receiver Arduino displayed the Hello World! message on the Serial Monitor every second as expected, indicating a successful data transfer from the sender Arduino and a functioning communication setup between the two Arduinos.

Discussion:

This experiment shows how to implement direct serial communication between two Arduinos. It also showed that the message was transmitted reliably using only two data connections (TX and RX) and shared ground. This setup shows how important correct pin connections and shared ground are, as any problem in either can prevent communication. Also, both Arduinos should be running at the same baud rate to make sure receipt and data transmission without mistakes. This experiment demonstrates that the Serial.println () function on sender and Serial.readString () on receiver handle information transfer and visualization in the Serial Monitor, which is shows that this experiment it can be a good example of inter-device communication.

Push Button & LED using 2 Arduinos

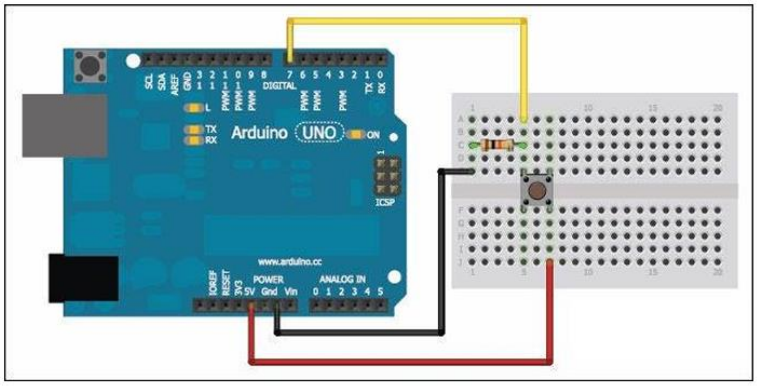

Figure 19: Arduino1 connected with Push button

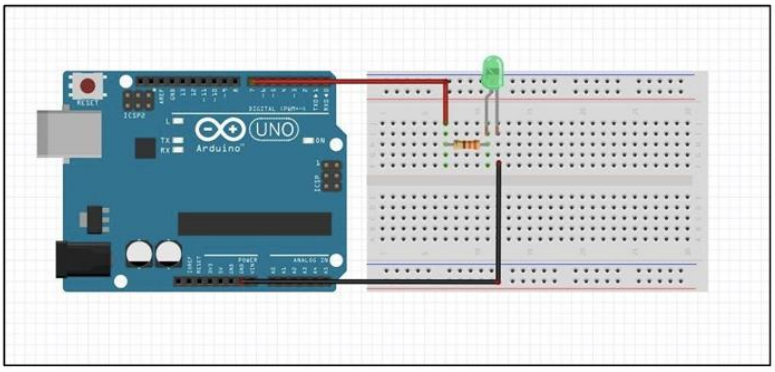

Figure 20: Arduino2 connected with Led

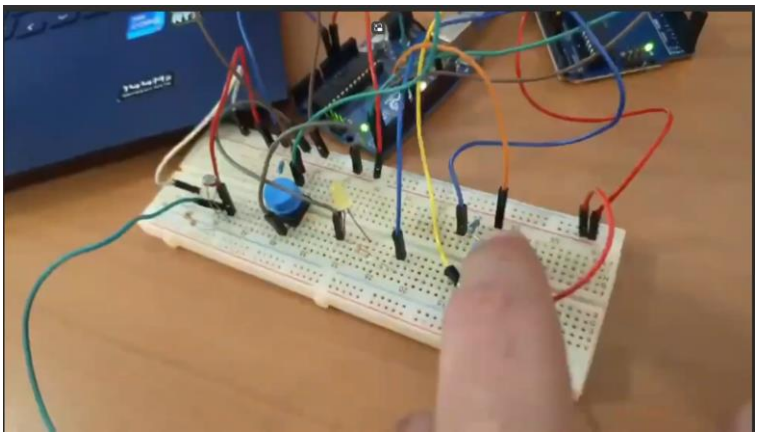

Figure 21: implementation of the circuit in lab

In this experiment, when the push button on Arduino 1 pressed it sends a 1 to Arduino 2 , which causes the led to turn on. The push button is connected to digital pin 7 on Arduino1 with a pull-down resistor and the led is connected to pin 7 on Arduino 2 with current limiting resistor. The TX pin of Arduino1 is connected to the RX pin of Arduino2 and also the RX pin of Arduino1 on the TX pin of Arduino2 (on common ground shared between the two Arduinos). The code on Arduino1 determines if the button is pressed and sends a "1" to Arduino2 with a delay to prevent repeated signals. Arduino2 then listens for incoming data and turns on the LED for 1 sec when it receives "1."

Results and Discussion

When push button Arduino1 was pressed, Arduino2 received value "1" and LED was activated for 1 second ,which shows that data transfer between the 2 Arduinos was successful. This resulted in a setup and code working correctly, where Arduino1 was sending data and Arduino2 was receiving and responding appropriately.

In Arduion1 it always check the state of the button so when the button is pressed , it sends the signal “1” through the function Serial.wirte(), then we have delay safe time to catch that signal from the receiver, so in arduino2 we ensure that there is any available data by the function Serial.available() so when the signal comes from the Arduion1 the variable receive data in arudino2 read the data , so if it’s 1 the lead will be high on pin7 , which show us

it shows the effectiveness of serial communication for simple inter-device signaling, and how we can use it in many applications.

You can verify this part by this video link:
https://drive.google.com/file/d/1pKhnoRpn2m3kR2yWPksj7NtAh2uQDhr8/view?usp=sharing

Part4: Visualization of serial communication using Serial Plotter

In this part we aim in this experiment to visualize the transmission of serial data as start , data, parity and stop bit using the serial plotter in Arduino IDE. It needed two arduino uno boards connected to communication in this experiment.

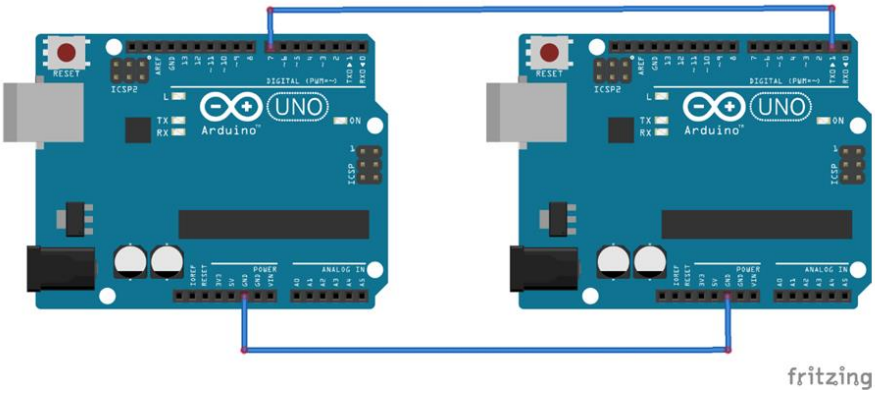

Figure 22: Arduino1 TX connected with Arduino2 PIN_7 with common GND

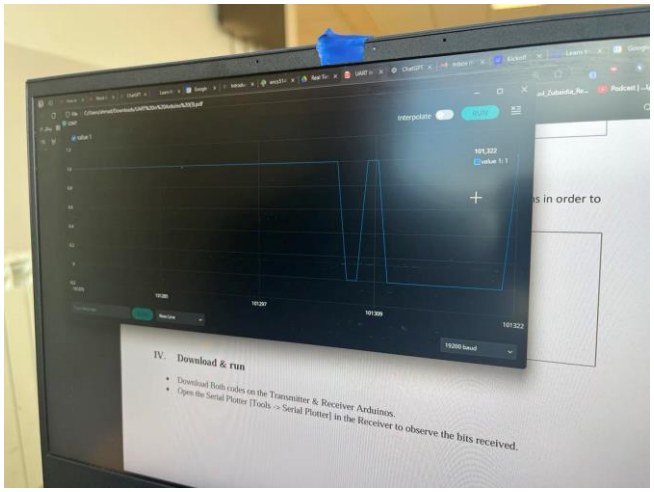

Figure 23 :Visualization of serial communication

To start the experiment the TX pin of Arduino1 is connected to digital pin 7 on Arduino2, ensuring both Arduinos share a common ground.

Then setting the sender (Arduino1), at a low baud rate of 300. Then In the loop(), the character 'A' is transmitted every 500 milliseconds using Serial.write('A'), which sends the character as a byte (41H in hexadecimal).

Meanwhile the receiver (Arduino2) is set to read the digital state of pin 7 at a higher baud rate of 19200. The code continuously reads the state of this pin using digitalRead(readPin) and prints the result to the Serial Monitor, which will be plotted in the Serial Plotter.

Results and Discussion

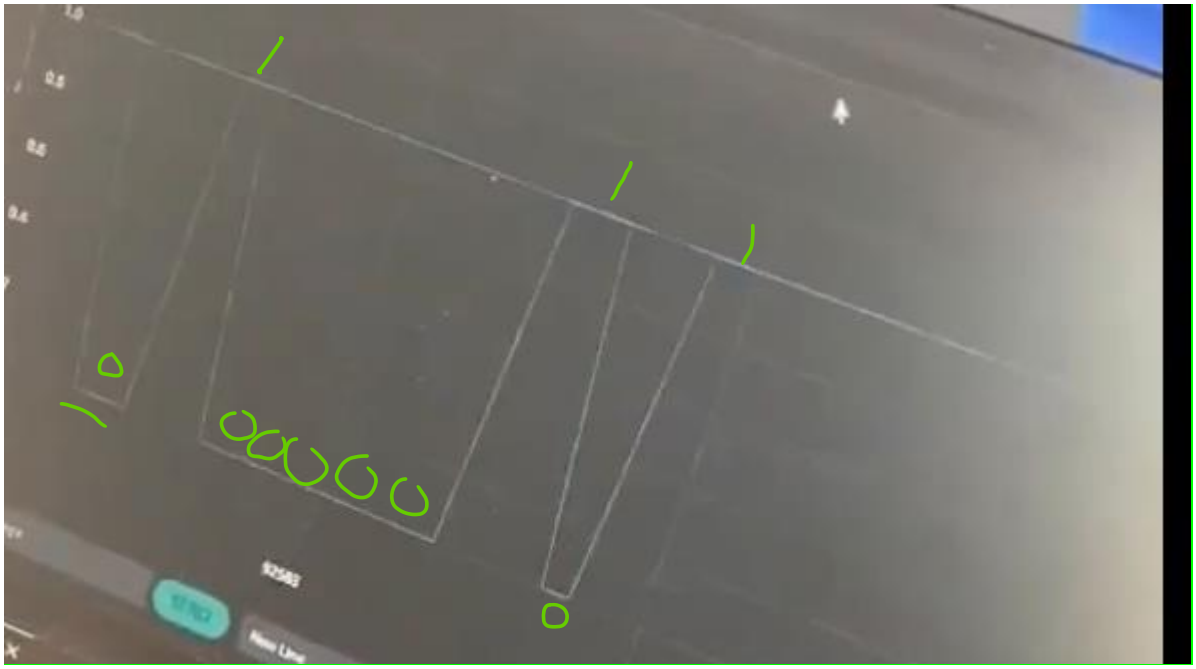

Figure 24: start data parity stop bits

This experiment visualizes the serial data transmission structure by sending bits of a character from one Arduino to another. In this experiment, the sender Arduino sends character 'A' at low baud rate of 300 so to make possible that every bit of information it be visible in the Serial Plotter. By settings the rates we can see the start, data, and stop bits in the transmission.

In asynchronous serial communication, information is transmitted as a sequence of bits framed by the start and stop bits. The start bit is a low signal (0), dropping from a high idle state to signal the start of data packet. Then the data bits follow the starting bit , For the character 'A' in the experiment 0x41 in hexadecimal the binary representation is 01000001. This particular sequence of bits is transmitted one by one in which 0 is a low pulse and 1 is a high pulse on the Serial Plotter as shown in the image above first the start bit from high to low then the data starts. The stop bit then sends the signal high after the data bits to notify or signal the end of the packet.

This visualization helps to understand serial data structure and how the Arduino treats change in signal states as binary data. Also, it’s noticeable that Slower baud rate makes each bit transition more visible, which show how asynchronous communication behave and how it works, and that provide a very good method for debugging and analyzing serial protocols.

To-do

In this to do we just added new button to the pin 8 at Arduino1 , also we added photo cell which give us the measure of how light level in the room , we connect it with to analog pin in order when we press the second button in the ardunio1 it change the value of the led on arduion2 and change the state of the led according to light sensor.

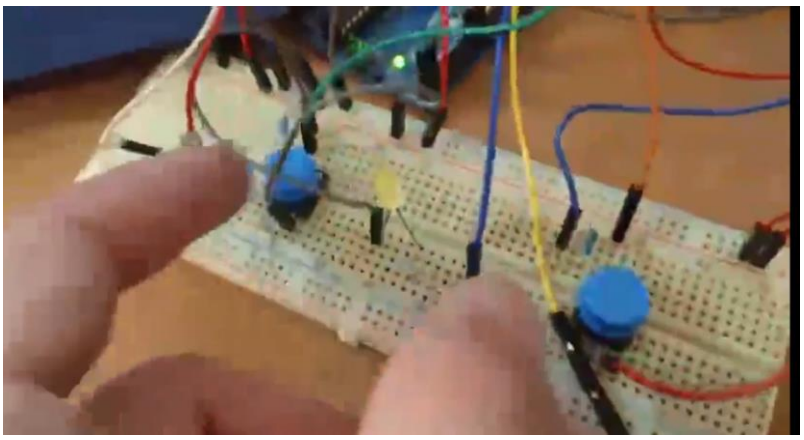

Figure 25: To do experiment 2

First the button 1 is connected to the pin 7 and button 2 is connected to the pin 8 , then we initialized barud rate to 9600 , and keep checking if anyone of the button is being pressed if the button 1 is pressed , the arudion1 will send the value “1” through serial.print to arduion2

And the arudion2 catch it by keep checking the seral. available () so when it comes value 1 it toggle the led ,but if the button 2 pressed the arduion1 will send 2 so the receiver will read the value of the led sensor then map it to value (0-255) then we will send this value to change the state of the led.

The result works perfectly as expected , but there was some delay when we press the buttons may be this because of debounce effect or to the problem of the button itself. But the step works perfectly.

By this experiment was informative it shows how we can use serial communication to have different functions by two different secrets, also it shows how important to connect the buttons on pull down resistor in order to have stable logic , also it shows to have perfect experience the buttons should be handled by interrupt and also handle the issue of debounce effect to get better results.

You can verify by this video link :
https://drive.google.com/file/d/1xSm8_sogQdD7NqFySn8aaTiD4PuEjAri/view?usp=sharing

All videos above here : RealTimeLab_EXP2_videos&pictures - Google Drive

Experiment 3

Part1: Writing the LCD to Arduino and I2C scanning

In this part, the LCD was connected to the Arduino through the SDA and SCL pins which are the default ones on the Arduino board which are A4 and A5 in order. Also, the VCC and GND pins on the LCD were connected to the VCC (5Volts) and GND on the Arduino Board as the in the below figure.

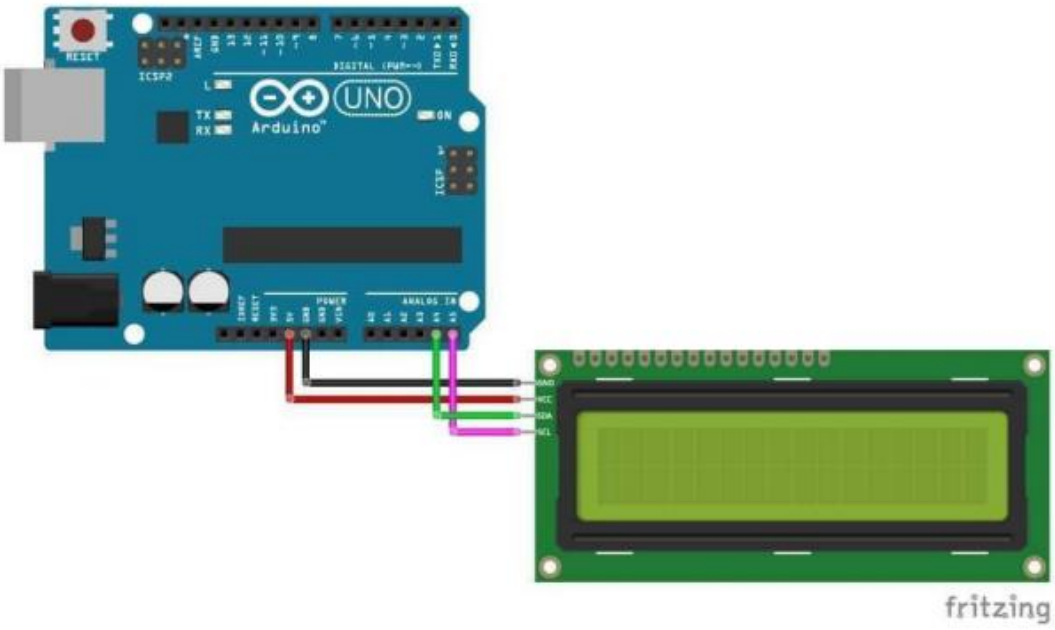

Figure 26: I2C module connection between Arduino and LCD

The software procedure started with installing the LiquidCrystal_I2C library by Marco Schwartz, then the code listed in the Appendix_Expirement3_part1 was uploaded in the IDE to discover the LCD address in order to prepare for the next part which has the part of displaying text on the LCD. The code consisted of two main parts; the setup and the loop parts. The setup mainly set the baud rate at 9600 then the loop part started looping through the 128 possible addresses of the I2C connection since the address has 8 bits so it could have 128 connection.

The main parts of the code were

wire.beginTransmission(address) and the **error = wire.endTransmission** that returned a value store in the error variable, each error value had a meaning so if the error was returned in zero value it means that there was an I2C device detected at the specific address given to the function of beginTransmission of the wire library. Also, if the error was equal to 4 that means that an unknown error was found at the address the transmission began with.

The address was found out to be 0x27 after uploading the code.

Part2: Display static text on the LCD

This part consisted of configuring the hardware part as in the previous part which is linked in the figure above. The LCD was connected to the Arduino UNO through the SDA and SCL pins which are corresponding to A4, A5 in order.

So, after finding the address of our LCD in the past part which was 0x27, this part involved writing text values on the LCD using the I2C communication through the LiquidCrystal_I2C library.

The software part included three main modules represented by initialization of the LCD library, the setup and the loop parts. Starting with the initialization; the address that we found which was 0x27 was the first parameter then the columns which was defined as 16 which is the number of characters we will write in a row, then the last parameter was the number of rows that will be displayed on the LCD. Then the second module is the setup which has the **lcd.init()** that initializes the LCD and **lcd.backlight()** that turn on the backlight so we can see the text presented on the screen.

Last module is the loop module which has the cursor set to the first column and the first row by this **lcd.setCursor(0,0)**. Then the text segment (“JAL TEAM”) was placed as a parameter to the **lcd.print()**. It was displayed for 1 second as shown in the figure below, and then the LCD was cleared through the statement of **lcd.clear()**. Then the same thing eas done for the other text that got displayed which consisted of our names(“Jana Ahmad Lana”) on the second row on the LCD after setting the cursor on the second row by the use of **lcd.setCursor(0,1)**, as shown in the figure below:

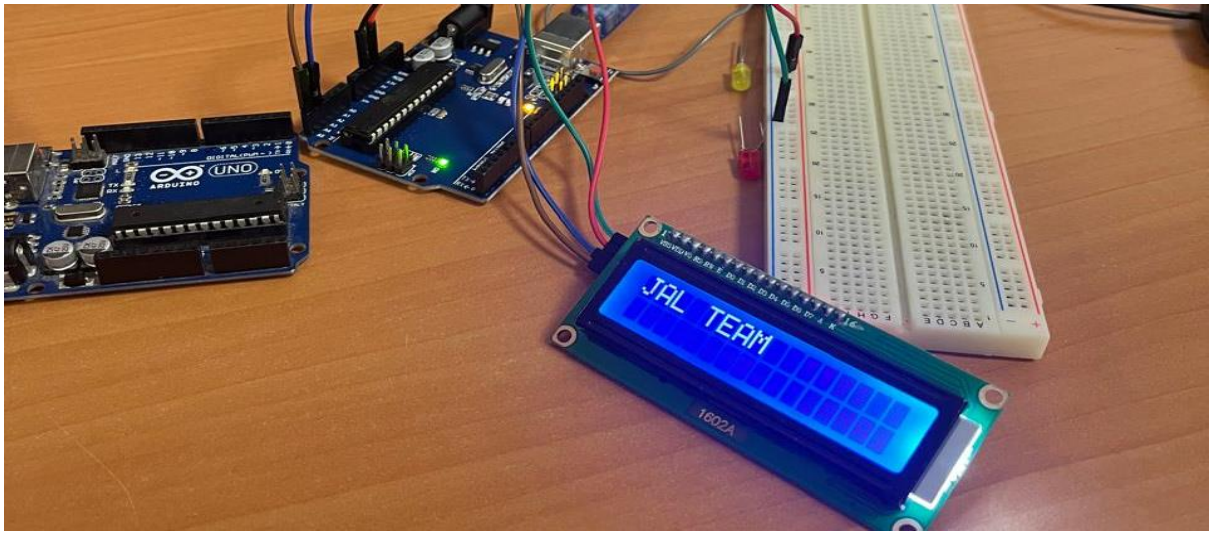

Figure 27: Displaying the first line on the LCD

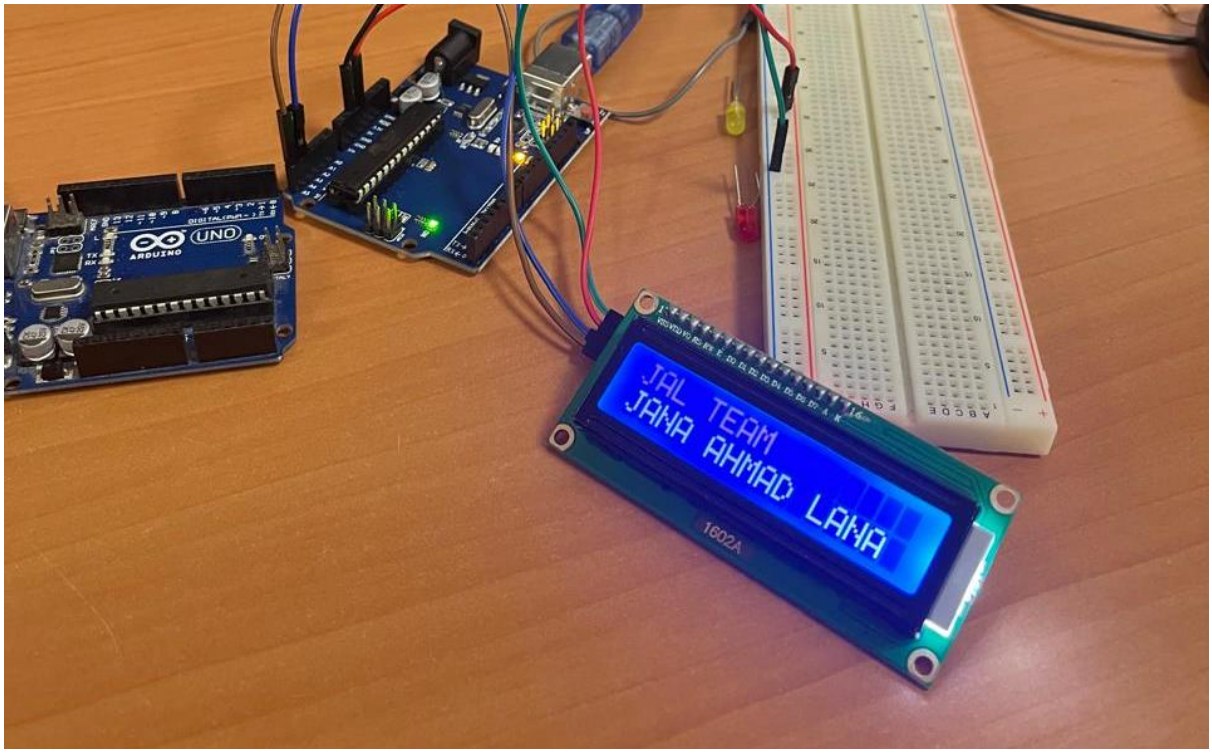

Figure 28 : Displaying the second line on the LCD

One challenge was when the first LCD was not giving any light and that told that it was broken so we got another one and it worked just fine.

Part3: I2C communication between 2 Arduinos

For this part, the hardware setup consisted of two Arduinos connected to each other through the SDA in the master with the SDA in the peripheral, also the SCL in the master with the SCL in the peripheral. And lastly connecting their grounds together as shown in the figure below:

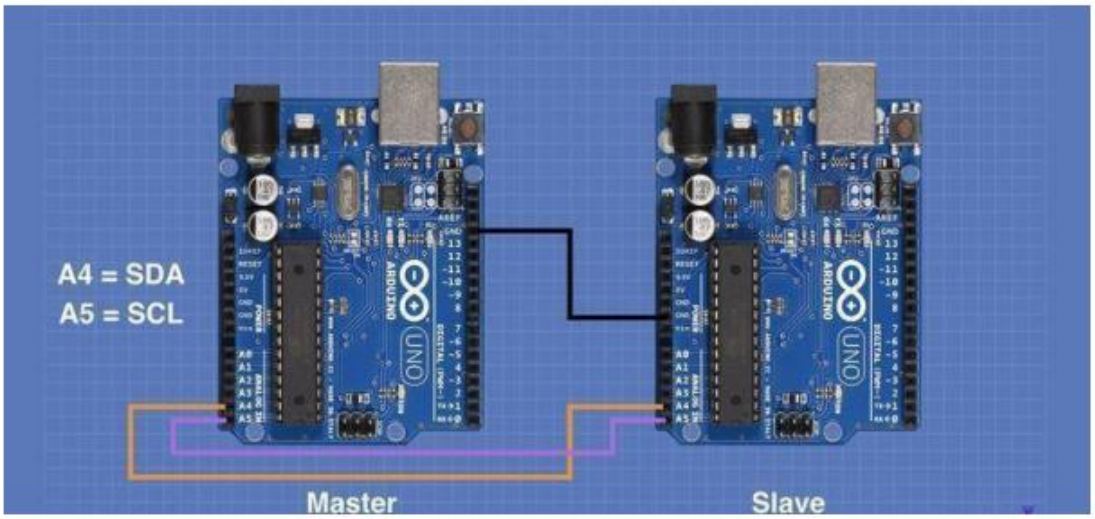

Figure 29: Hardware setup for communication between master and peripheral

The software part included two segments of codes which are listed in the Appendix_Expirement3_part3. The first segment had the master logic which was reading through the I2C connection with the peripheral. The master baud rate was set up to 9600 and printed master sleeping for 2 seconds then printed go to the serial monitor to indicate that it entered the loop part of the code and started reading from the peripheral.

Then, the loop part had **Wire.requestFrom(2,1)** which started to request data from the peripheral, the 2 is the I2C address specified by the **Wire.begin(2)** which is in the setup part of the peripheral, the 1 is the number of bytes requested by the master which is one in here. The request from will execute a function called requestEvent() in the peripheral which will be discussed below. Then if there is an available byte written on the wire which is checked by **Wire.available(),** the master will read the character using **Wire.read()** and print it to the serial monitor.

The second segment had the peripheral arduino which started the I2C communication with the **Wire.begin(2)** statement which initiates the communication at address 0x02. Then the function that will be executed whenever the master requests from the peripheral was registered as an event using the code segment of **Wire.onRequest(requestEvent).**

The function starts with the character ‘0’ and then increments the character by 1 till it reaches to ‘z’, it goes back to zero to set it back for another call from the master. For each character the statement of Wire.write(c++) was executed to send the character to the master.

The expected results were from 0 to 9 , from A to Z and from a to z and some punctuations and that was done as in the video in link :

https://drive.google.com/drive/folders/1KJx2IezaN9AT1A4WB-wORvU9D9KUyENQ

And the results that we got matched the expected one correctly.

0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz

Part4: I2C communication between 2 Arduinos and LCD

The schematic in the last Figure 13 was recreated, the SDA, SCL and the ground of the two arduinos were connected together between the master and the peripheral. Then, the **LCD** was connected to the **master** device after flipping the arduino and locating the SCL and SDA pins and connecting them to the corresponding pins on the LCD. Then, the **potentiometer** got connected to the **peripheral** device on the A0 pin. And that actually was done through connecting the left side to the ground and the right side to the VCC and the middle part to the A0 pin in the Arduino.

As a preparation for this part, the circuit was configured on tinkercad for the peripheral in order to revise how the potentiometer is connected to give a proper analog values as shown in the picture below:

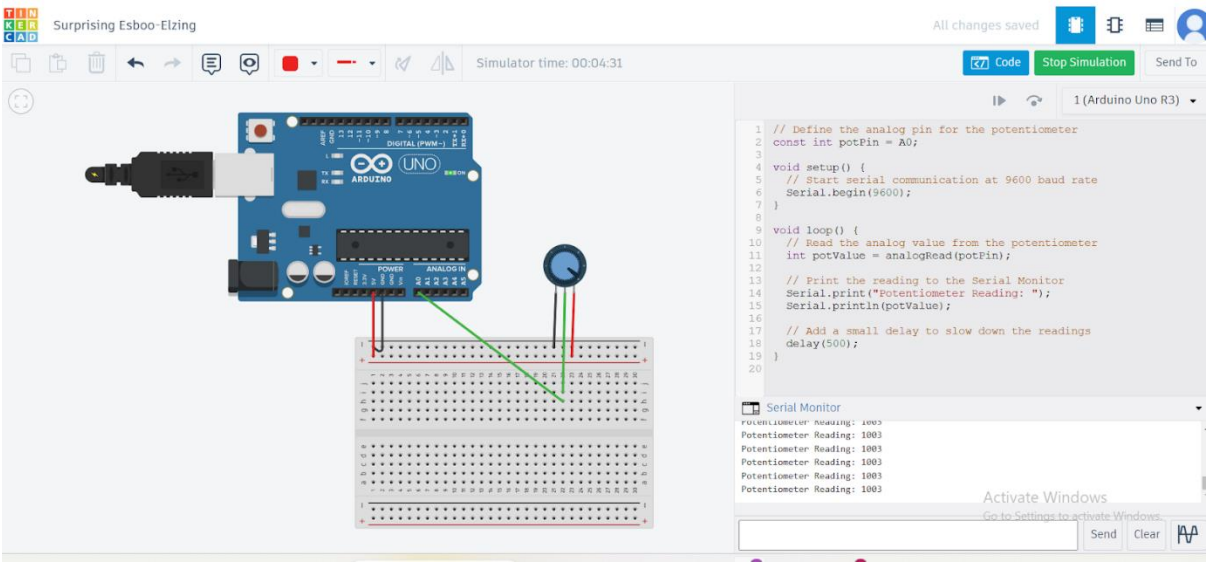

Figure 30: Connecting the potentiometer on the tinkerkad

For the peripheral software part, the setup module is configured of initializing the I2C communication at address 0x50 using **Wire.begin(50)** and registering the event of the function **requestEvent().** This function is executed whenever the master requests data from the peripheral it reads the analog value of the A0 pin which is connected to the potentiometer.

The possible values of the potentiometer is from 0 to 1023, meaning that it will be at maximum 10 bits which doesn't fit in one byte only. So we need to carry the 10 bits in two times to the master and that is done through **Wire.write(value >> 8); Wire.write(value & 0x00FF);**

The first statement shifts the most significant bits to the right by 8 positions so the most significant 2 bits will be transferred to the master by this point . Then the original 10 bits will be anded with 0x0011111111 which will get the least significant 8 bits and they will be transferred to the master and by that the full 10 bits will be received successfully.

Now, for the master software part, first the LCD was configured the same as in part2 of this experiment that is summarized by configuring the address of the LCD, the number of rows and columns to present on the screen, then in the setup part; the baud rate was configured and the lcd was initialized and the backlight was turned on.

Then in the loop part, the data is requested from the peripheral using **Wire.requestFrom(50,2).** 50 indicates from the address of the peripheral that will have the I2C communication with , the 2 indicates for the number of bytes that will be transferred through the channel of the I2C communication. Then the 10 bits that were read through **Wire.read()** transferred from the peripheral is combined in the master by **res = ((MSB << 8) | LSB);** which shifts the most significant 2 bits back to the left and then its orred with the remaining bits in the least significant bits. Lastly, the result was printed on the LCD after setting the cursor at the first row and the first column **lcd.setCursor(0, 0);**

The results matched the expected theoretical results as shown in this video.

https://drive.google.com/drive/folders/1KJx2IezaN9AT1A4WB-wORvU9D9KUyENQ

Part 5: I2C communication between Arduino and Temperature sensor (LM75B)

This part involved communication between Arduino and LM75B sensor using I2C, the first step was finding out the address of the temperature sensor which was found at 0x48 using the first part of the experiment.

The hardware configuration was done according to the figure below:

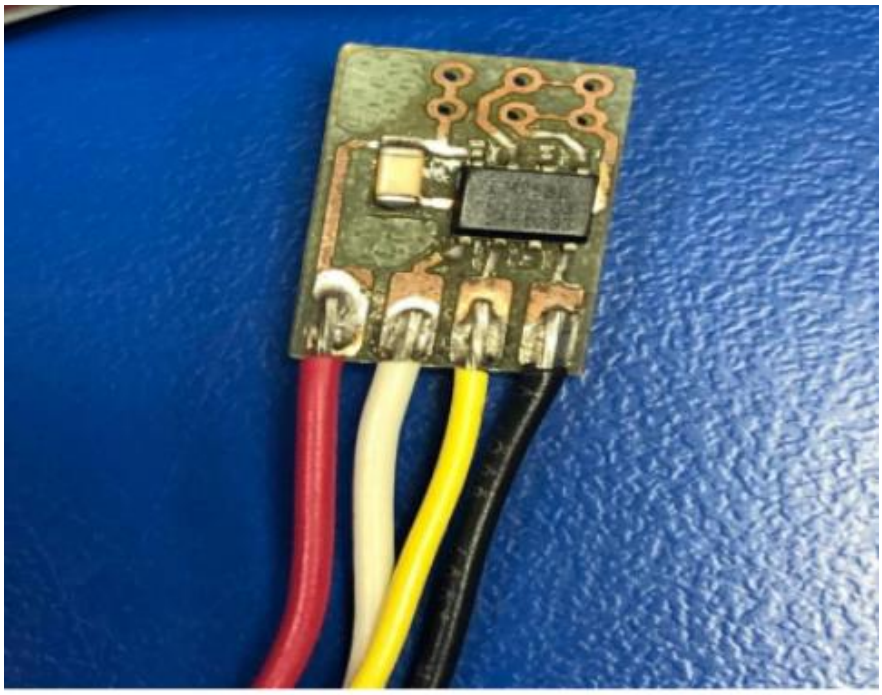

Figure 31: LM75B wires

LM75B	Arduino
Black wire	GND
Red wire	VCC
White wire	SDA
Yellow wire	SCL

Table 5 Reference of connecting the temperature sensor to the Arduino :

After connecting each wire to the corresponding Arduino pin according to the table above, the software part was configured through the code listed in Appendix_Expirement3_part5.

The main part of the code is in the function of **getTempreture().** It starts with requesting 2 bytes

from the address of the temperature sensor which was found at 0x48 using the statement of **Wire.requestFrom(LM75BAddress, 2).** The 2 bytes are needed because it reads 12 bits, the most significant byte of the temperature sensor reading was read through **byte MSB = Wire.read()** and the least significant byte was also read afterwards through **byte LSB = Wire.read()**.

int TemperatureSum = ((MSB << 8) | LSB) >> 5;

The temperature sum is concatenated in an integer which is 16 bits in total. So, the most significant bits of the 12 bits reding are shifted 8 bits to the left. That creates enough space for the least significant which are 4 actual bits from the 12 reading bits and another 4 since its defined as a byte to lay down next to the most significant ones which is done through the orring between them. The last step was shifting the sum to the right by 5 bits to make it again 12 bits since the first 4 bits were never used and they were just padded because the reading of the least significant bits were read as a full byte. Then the temperature sum was multiplied by 0.0625 which is a specific for the LM75B sensor's resolution; each bit of the 12bits represent 0.0625 degrees in Celsius according to this equation **float celsius = TemperatureSum * 0.0625;**

Finally, the code prints the temperature in Celsius and also convert it to Fahrenheit. The experiment was perfectly done and met the expectations since the temperature reading was changing when our hands were placed on top of it so it would give extra heat to the sensor and it read it successfully as shown in this video:

https://drive.google.com/drive/folders/1KJx2IezaN9AT1A4WB-wORvU9D9KUyENQ

TODO:

The todo involved an improvement of the 4th part of the experiment which included two Arduinos; the master had the LED and the LCD while the peripheral had the potentiometer along with the reset button.

The Led was connected to pin 9 on the master because it had the feature of PWM writing on it. While the button pin was connected on the pin 7 of the peripheral Arduino.

The peripheral had the reading process of the potentiometer as in part4 which is summarized by `analogRead(A0)` **and transferring the values of the 10 bits of the potentiometer through** `Wire.write(value >> 8);` `Wire.write(value & 0x00FF);` and had an extra configuration

to read the push button value through **digitalRead(BUTTON_PIN)** and send the value 1 to the master when its pressed though `Wire.write(buttonState);`

The master on the other hand requested 3 bytes from the peripheral which are 2 for the analog reading and 1 for the button state through `Wire.requestFrom(50, 3);`and then received the values of the push button through `byte buttonState = Wire.read();` and the potentiometer through

```
byte MSB = Wire.read();    byte LSB = Wire.read();
```

and mapped them to a logic to control the intensity of the LED according to the value of the potentiometer after printing its value to the LCD . and that was executed though `int brightness = map(res, 0, 1023, 0, 255);.` while checking whenever the value of the push button is one which means its pressed down it would reset the LCD by `lcd.clear();`

.

The results were successfully obtained after configuring the push button correctly on the board according to this

https://drive.google.com/drive/folders/1KJx2IezaN9AT1A4WB-wORvU9D9KUyENQ

we faced a challenge that we needed to change the function that writes the value of the led as an analog value not digital. But at the end we noticed that and it worked perfectly.

Disclosure of AI tools

Experiment 1

I used Chat GPT to understand the bouncing effect and make technical ideas clearer. I also relied on Grammarly and ChatGPT to check grammar and to paraphrase sentences for making all things clear. It also helped me to summarize the introduction and conclusion statements which are related to the first experiment.

Experiment 2

Used AI in mentioning what is the component for the some of the procedure. Also used in writing and re arranging the code in the appendix

Experiment 3

[AI] Range of addresses for the temperature sensor I2C addresses. Understanding concepts in the theory

Conclusion

In conclusion, the first experiment demonstrated using Arduino for light detection and real-time response through interrupts. We achieved the expected results by implementing a photocell and managing events with hardware and software interrupts. Through testing, issues with timing and sensor sensitivity were resolved. Future developments might speed up reaction times and increase the usage of LDR sensor to facilitate more engagement.

Moving to experiment 2, it demonstrated the fundamentals of UART serial communication and its configuration. It was found that for Successful communication between Arduinos and between an Arduino and a PC, it requires a matching between both devices at baud rates and also in the common ground, without these, data transmission fails, resulting in loss of data. also, the push button at the experiment 3 and at the to-do showed the importance and the inter-device communication by pressing a button on one Arduino to turn on an LED on the other, also the experiments showed how the structure of the data packet included start, data, and stop bits by visualizing it on the serial plotter.

To conclude the deverbals of Experiment 3, the I2C communication protocol was understood by first figuring out the addresses of the devices such as the LCD, and then transferring data to the LCD, then the communication protocol was applied between two Arduinos and lastly, we were able to understand the principal concept of the temperature sensor reading process. The three experiments overall highlighted the communication between the computer and the Arduino and other devices such as LDR, potentiometer, leds, push buttons, LCDs and how they can be connected to each other through either UART which is a physical connection through RX and Tx pins or through a full communication protocol which is the I2C.

Applying the concepts that we learned theoretically in the interface course was the most entertaining part and all the expected results were successfully obtained in the experiments as shown in the videos regardless to the simple challenges encountered that were a corruption in a device such as a led or a breadboard, they were all under control with the debugging skills with Eng.Rania(thank you!).

References

[1] https://docs.arduino.cc/learn/starting-guide/whats-arduino/ (2/11/2024, 1:20)

[2] https://en.wikipedia.org/wiki/Arduino_Uno(2/11/2024, 1:45)

[3] https://www.circuito.io/blog/arduino-uno-pinout/(2/11/2024, 2:10)

[4] https://www.circuitschools.com/what-is-arduino-how-it-works-and- what-you-can-do-with-arduino/(2/11/2024, 2:40)

[5] https://www.watelectrical.com/photocell/(3/11/2024, 1:32)

[6] https://learn.adafruit.com/photocells?view=all(3/11/2024, 2:15)

[7] Lab Manual

[8] https://www.engineersgarage.com/arduino-interrupts(3/11/2024, 4:30)

[9] https://microcontrollerslab.com/arduino-timer-interrupts-tutorial/(3/11/2024, 5:10)

[10] https://projecthub.arduino.cc/ (4/11/2024, 7:28)

[11] "circuitbasics," [Online]. Available: https://www.circuitbasics.com/basics-uart-communication/.

[12] "arduino," [Online]. Available: https://reference.arduino.cc/reference/en/language/functions/communication/serial/.

[13] I2C Primer: What is I2C? (Part 1) | Analog Devices

[14]Welcome to Real Digital

[15] I2C Primer: What is I2C? (Part 1) | Analog Devices

Appendix

Experiment 1 Codes

Part1: Photocells

```
int photocellPin = 0;         // the cell and 10K pulldown are connected to A0
int photocellReading;         // the analog reading from the sensor divider
int LEDpin = 11;              // connect Red LED to pin 11 (PWM pin)
int LEDbrightness;            // brightness level for the LED

void setup(void) {
  // We'll send debugging information via the Serial monitor
  Serial.begin(9600);
}

void loop(void) {
  // Read the analog value from the photocell
  photocellReading = analogRead(photocellPin);
  Serial.print("Analog reading = ");
  Serial.print(photocellReading); // the raw analog reading

  // Set light levels based on the photocell reading
  if (photocellReading < 200) {     // adjust the threshold as needed
    Serial.println(" - Dark");
  } else if (photocellReading < 700) {
    Serial.println(" - Light");
  } else {
    Serial.println(" - Bright");
  }

  // LED gets brighter the darker it is at the sensor
  // invert the reading from 0-1023 back to 1023-0
  photocellReading = 1023 - photocellReading;

  // Map 0-1023 to 0-255 since that's the range analogWrite uses
  LEDbrightness = map(photocellReading, 0, 1023, 0, 255);
  analogWrite(LEDpin, LEDbrightness);

  // Delay for readability in the Serial Monitor (adjust as needed)
  delay(1000);
}
```

Part2: Hardware Interrupts

```
int pin = 13;
volatile int state = LOW;
unsigned long lastDebounceTime = 0; // Last time the interrupt was triggered
unsigned long debounceDelay = 50;   // Debounce delay in milliseconds

void setup() {
  pinMode(pin, OUTPUT);
  attachInterrupt(0, blink, CHANGE); // Attach interrupt to pin 2 (interrupt 0) on
state change
}

void loop() {
  digitalWrite(pin, state); // Write the current state to pin 13 (LED on or off)
}

void blink() {
  unsigned long currentTime = millis(); // Get current time in milliseconds

  // Only toggle the state if the debounce delay has passed since the last interrupt
  if (currentTime - lastDebounceTime > debounceDelay) {
    state = !state;                     // Toggle the LED state
    lastDebounceTime = currentTime;      // Update the last debounce time
  }
}
```

Part3: Software Interrupts

```
// Timer1 overflow interrupt example
#define ledPin 13
void setup()
{
pinMode(ledPin, OUTPUT);
// initialize timer1
noInterrupts(); // disable all interrupts
TCCR1A = 0;
TCCR1B = 0;
TCNT1 = 34286; // preload timer 65536-16MHz/256/2Hz
TCCR1B |= (1 << CS12); // 256 prescaler
TIMSK1 |= (1 << TOIE1); // enable timer overflow interrupt
interrupts(); // enable all interrupts
}
ISR(TIMER1_OVF_vect) // interrupt service routine that wraps a user
{
TCNT1 = 34286; // preload timer
```

```
digitalWrite(ledPin, digitalRead(ledPin) ^ 1);
}
void loop()
{
}
```

Experiment 2 Codes

Part1: Basic communication between Arduino & PC

```
byte byteRead;
void setup() {
// Turn the Serial Protocol ON
Serial.begin(9600);
}
void loop() {
/*  check if data has been sent from the computer: */
if (Serial.available()) {
/* read the most recent byte */
byteRead = Serial.read();
/*ECHO the value that was read, back to the serial port. */
Serial.write(byteRead);
}
}
```

Part2: Basic communication between 2 Arduinos

Sender Code:

The sender will be responsible of sending "Hello World!" to the receiver Arduino. First, we should specify the serial communication Setup [Baud rate, stop bits, …] and then send!

```
void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);
}

void loop() {
  Serial.println("Hello World!"); //Write the serial data
  delay(1000);
}
```

Receiver Code:

The receiver will be responsible of receiving the data and then displaying it. First, we should specify the serial communication Setup [Baud rate, stop bits, …] and then read the data!

```
void setup() {
  // Begin the Serial at 9600 Baud
  Serial.begin(9600);
}

void loop() {
   if (Serial.available()) {
    String data = Serial.readString();//Read the serial data and store in var
    Serial.println(data);//Print data on Serial Monitor
   }
```

Figure 32: code from part 2

Part3: Push Button & LED using 2 Arduinos

```
int ledPin = 7; // Pin where the LED is connected
int receivedData;

void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);

  // Initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Check if data has been sent from Arduino1
  if (Serial.available() > 0) {
    // Read the incoming byte
    receivedData = Serial.read();
```

```
    // Check if 1 is sent, and turn on the LED if yes
    if (receivedData == 1) {
      digitalWrite(ledPin, HIGH);  // Turn on LED
      delay(1000);  // Keep LED on for 1 second
      digitalWrite(ledPin, LOW);   // Turn off LED
    }
  }
}

int buttonPin = 7; // Pin where the push button is connected
int buttonState = 0;

void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);

  // Initialize the push button pin as an input
  pinMode(buttonPin, INPUT);
}

void loop() {
  // Check if the push button is pushed
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    // Send 1 to Arduino2 if pushed
    Serial.write(1);
    delay(200); // Debounce delay to avoid multiple readings
  }
}
```

Part4: Visualization of serial communication using Serial Plotter

III. Code

Sender Code:

The sender will be responsible of sending a character to the receiver Arduino. First, we have to setup the Baud Rate to be very small [for example, 300] and then sending a character with a delay.

```
void setup() {
  // Setup the Serial at 300 Baud
  Serial.begin(300);
}

void loop() {
  Serial.write('A'); //Write the character A => will be transmitted as Byte [41H]
  delay(500);
}
```

Receiver Code:

The receiver will be responsible of reading the data sent using digitalRead with one of the digital pins in order to plot the serial data as bits. A high Baud rate was used in order to capture the bits.

```
int readPin = 7;
void setup() {
  // Begin the Serial at 19200 Baud
  Serial.begin(19200);
  pinMode(readPin, INPUT);
}

void loop() {
  Serial.println(digitalRead(readPin));
}
```

Figure 33: code for experiment 4

To do Experiment 2 :

```
int buttonPin = 7; // Pin where the push button is connected
int buttonPin2 = 8 ;

int buttonState = 0;
int buttonstate2 = 0;

void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);

  // Initialize the push button pin as an input
  pinMode(buttonPin, INPUT);
}

void loop() {
  // Check if the push button is pushed
```

```
    buttonState = digitalRead(buttonPin);
    buttonstate2 = digitalRead(buttonPin2);


  if (buttonState == HIGH) {
    // Send 1 to Arduino2 if pushed
    Serial.print('1');
    delay(200); // Debounce delay to avoid multiple readings

  }

  if (buttonstate2 == HIGH ){

    Serial.print('2');
    delay(200); // Debounce delay to avoid multiple readings
  }

}

int ledPin = 13;         // Pin where the LED is connected
int photoCellPin = A0;   // Analog pin where the photoresistor is connected
int ledBrightness = 0;   // Variable to store the LED brightness
bool ledState = LOW;     // State to keep track of the LED (on/off)

void setup() {
  // Setup the Serial at 9600 Baud
  Serial.begin(9600);

  // Initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Check if there's any data available on the Serial
  if (Serial.available() > 0) {
    // Read the incoming byte from Arduino1
    char incomingByte = Serial.read();

    // If '1' is received, toggle the LED
    if (incomingByte == '1') {
      ledState = !ledState;       // Toggle the LED state
      digitalWrite(ledPin, ledState ? HIGH : LOW); // Update the LED
    }

    // If '2' is received, read the photoresistor value and adjust the LED brightness
    if (incomingByte == '2') {
      int sensorValue = analogRead(photoCellPin);      // Read photoresistor value
      ledBrightness = map(sensorValue, 0, 1023, 0, 255); // Map to 0-255 for PWM
      analogWrite(ledPin, ledBrightness);              // Set LED brightness
    }
```

```
  }
}
```

Experiment 3 Codes

Part1:Writing the LCD to Arduino and I2C scanning

```
#include <Wire.h>
void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("\nI2C Scanner");
}
void loop() {
  byte error, address;
  int nDevices;
  Serial.println("Scanning...");
  nDevices = 0;
  for(address = 1; address < 127; address++ ) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address<16) {
        Serial.print("0");
      }
      Serial.println(address,HEX);
      nDevices++;
    }
    else if (error==4) {
      Serial.print("Unknow error at address 0x");
      if (address<16) {

      Serial.print("0");
      }
    Serial.println(address,HEX);
    }
  }
  if (nDevices == 0) {
    Serial.println("No I2C devices found\n");
```

```
  }
  else {
    Serial.println("done\n");
  }
  delay(5000);
}
```

Part2: Display static text on the LCD

```
#include <LiquidCrystal_I2C.h>
// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;

LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);

void setup(){
  // initialize LCD
  lcd.init();
  // turn on LCD backlight
  lcd.backlight();
}
void loop(){
  // set cursor to first column, first row
  lcd.setCursor(0, 0);
  // print message
  lcd.print("JAL TEAM");
  delay(1000);
  // clears the display to print new message
  lcd.clear();
  // set cursor to first column, second row
  lcd.setCursor(0,1);
  lcd.print("JANA AHMAD LANA”);
  delay(1000);
  lcd.clear();
}
```

Part3: I2C communication between 2 Arduinos

```
// master side code
```

```
#include <Wire.h>
void setup()
{
  Wire.begin(); // join i2c bus (address optional for master)
  Serial.begin(9600); // start serial for output
  Serial.print("master sleeping...");
  delay(2000);
  Serial.println("go");
}
void loop()
{
  Wire.requestFrom(2, 1); // request data from slave device #2
  while(Wire.available())
  {
    char c = Wire.read(); // receive a byte as character
    Serial.print(c); // print the character
  }
{
  delay(100);
}

// slave side code
#include <Wire.h>
void setup()
{
  Wire.begin(2); // join i2c bus with address #2
  Wire.onRequest(requestEvent); // register event
  Serial.begin(9600); // start serial for output
}
void loop()
{
  delay(100);
}
// function that executes whenever data is received from master
// this function is registered as an event, see setup()
void requestEvent()
{
  static char c = '0';
  Wire.write(c++);
  if (c > 'z')
```

```
    c = '0';
}
```

Part 2.5: I2C communication between Arduino and Temperature sensor (LM75B)

```
#include <Wire.h>

int LM75BAddress = 0x48;
void setup() {
  Serial.begin(9600);          // Start serial communication at 9600 baud
  Wire.begin();                 // Start I2C communication
  Serial.println("communication start");
}

void loop() {
  float celsius = getTemperature();  // Get temperature in Celsius
  Serial.print("Celsius: ");
  Serial.println(celsius);

  float fahrenheit = (1.8 * celsius) + 32;  // Convert Celsius to Fahrenheit
  Serial.print("Fahrenheit: ");
  Serial.println(fahrenheit);

  delay(200);  // Delay for readability; adjust as necessary
}

float getTemperature() {
  Wire.requestFrom(LM75BAddress, 2);  // Request 2 bytes from the LM75B
sensor

  byte MSB = Wire.read();  // Most Significant Byte
  byte LSB = Wire.read();  // Least Significant Byte

  // Convert the 12-bit reading to Celsius
  int TemperatureSum = ((MSB << 8) | LSB) >> 5;

  // Convert two's complement for negative temperatures if needed
  if (TemperatureSum > 0x7FF) {  // If the value is negative
    TemperatureSum |= 0xF000;    // Extend the sign bit
  }
```

```
  float celsius = TemperatureSum * 0.0625;  // Each bit represents 0.0625
degrees
  return celsius;
}
```

Part4: I2C communication between 2 Arduinos and LCD

```
// master side code
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;
// set LCD address, number of columns and rows
// if you don't know your display address, run an I2C scanner
sketch
LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);
void setup()
{
  Wire.begin(); // join i2c bus (address optional formaster)
  Serial.begin(9600); // start serial for output
  Serial.print("master sleeping...");
  delay(2000);


  // initialize LCD
  lcd.init();
  // turn on LCD backlight
  lcd.backlight();
}
void loop()
{
Wire.requestFrom(50, 2);

  // request data from slave device #2

  int res;//result from I2c reading
  byte MSB = Wire.read(); /* receive a byte MSB(NOTE: the
  function is blocking, so it would not continue the code until a
  byte reach)*/
```

```
  byte LSB = Wire.read(); /* receive a byte LSB(NOTE: the
  function is blocking, so it would not continue the code until a
  byte reach)*/
  res = ((MSB << 8) | LSB);
  Serial.print("MSB = ");
  Serial.print(MSB);
  Serial.print(" , LSB = ");
  Serial.print(LSB);
  Serial.print(" , analog value = ");
  Serial.println(res); // print the analog value

  // set cursor to first column, first row
  lcd.setCursor(0, 0);

  // print message
  lcd.print("Reading = ");
  lcd.print(res);
  delay(1000);
  lcd.clear();
}

// slave side code
#include <Wire.h>
void setup()
{
  Wire.begin(50); // join i2c bus with address #2
  Wire.onRequest(requestEvent); // register event
  Serial.begin(9600); // start serial for output
}
void loop()
{
  delay(100);
}
// function that executes whenever data is received from master
// this function is registered as an event, see setup()
void requestEvent()
{
  int value = analogRead(A0);
  Wire.write(value >> 8); // send the MSB
  Wire.write(value & 0x00ff); // send the LSB
```

```
}
```

TODO

Master code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define LED_PIN 9  // Define pin 7 for the LED

// set the LCD number of columns and rows
int lcdColumns = 16;
int lcdRows = 2;

// set LCD address, number of columns and rows
LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);

void setup()
{
  Wire.begin();             // join I2C bus (address optional for master)
  Serial.begin(9600);       // start serial for output
  Serial.print("master sleeping...");
  delay(2000);
  Serial.println("go");

  // initialize LCD
  lcd.init();
  // turn on LCD backlight
  lcd.backlight();

  // Initialize the LED pin as output
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
```

```
  // Request 3 bytes from the slave (2 for analog reading, 1 for the button state)
  Wire.requestFrom(50, 3);

  int res;  // Variable to hold the result from I2C reading
  byte MSB = Wire.read();  // Receive the MSB byte (blocking until received)
  byte LSB = Wire.read();  // Receive the LSB byte (blocking until received)
  byte buttonState = Wire.read();  // Receive the button state (1 for pressed, 0 for
not pressed)

  // Combine MSB and LSB to get the analog value
  res = ((MSB << 8) | LSB);

  // Print the analog value and button state to the Serial Monitor
  Serial.print("MSB = ");
  Serial.print(MSB);
  Serial.print(" , LSB = ");
  Serial.print(LSB);
  Serial.print(" , analog value = ");
  Serial.print(res);
  Serial.print(" , button state = ");
  Serial.println(buttonState);

  // Display the analog value on the LCD
  lcd.setCursor(0, 0);          // Set cursor to first column, first row
  lcd.print("Reading = ");       // Print the label "Reading ="
  lcd.print(res);                // Print the analog value

  // Check if the button is pressed (buttonState == 1)
  if (buttonState == 1)
  {
    lcd.clear();  // Clear the LCD when the button is pressed
  }

  // Map the analog value `res` (0-1023) to the PWM range (0-255)
  int brightness = map(res, 0, 1023, 0, 255);

  // Set the LED brightness using PWM
  analogWrite(LED_PIN, brightness);

  delay(1000);  // Wait for 1 second before the next loop
}
```

SLAVE CODE

```
#include <Wire.h>
```

```
#define BUTTON_PIN 7  // Define the pin number for the push button

void setup()
{
  Wire.begin(50);  // Join I2C bus with address #50
  Wire.onRequest(requestEvent);  // Register event for when master requests data
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Set pin 7 as input for the push button
(with internal pull-up resistor)
  Serial.begin(9600);  // Start serial communication for debugging
}

void loop()
{
  delay(100);  // Short delay in the loop
}

// Function that executes whenever data is requested from master
void requestEvent()
{
  int value = analogRead(A0);  // Read the analog value from pin A0

  Wire.write(value >> 8);  // Send the most significant byte (MSB)
  Wire.write(value & 0x00FF);  // Send the least significant byte (LSB)

  int buttonState = digitalRead(BUTTON_PIN);  // Read the push button state (HIGH or
LOW)
  Serial.println(buttonState);


  // Send the button state (either 1 for HIGH or 0 for LOW)
  Wire.write(buttonState);  // Send the push button state to the master
}
```